



Universidad de Castilla-La Mancha

Escuela Superior de Ingeniería Informática de Albacete

Escuela Superior de Informática de Ciudad Real

Trabajo Fin de Máster

Máster Universitario en Big Data y Computación en la Nube

Diseño y configuración de un modelo de metadatos en OpenMetadata conforme al estándar DCAT-AP para la interoperabilidad de catálogos de datos

Alonso Marcos Muñoz

Julio, 2026

TRABAJO FIN DE MÁSTER

Máster Universitario en Big Data y Computación en la Nube

Diseño y configuración de un modelo de metadatos en OpenMetadata conforme al estándar DCAT-AP para la interoperabilidad de catálogos de datos

Autor: Alonso Marcos Muñoz

Tutor: Fernando Gualo Cejudo

Codirector: Antonio Labian Moya

Julio, 2026

*A mi familia y a quienes me han acompañado durante el máster,
por la paciencia y el apoyo que han hecho posible
llegar hasta este trabajo.*

Declaración de autoría

Yo, Alonso Marcos Muñoz , con DNI 06281718G , declaro que soy el único autor del Trabajo Fin de Máster titulado “Diseño y configuración de un modelo de metadatos en Open-Metadata conforme al estándar DCAT-AP para la interoperabilidad de catálogos de datos”. El trabajo se ha realizado de forma individual y el material procedente de terceros se cita o atribuye de acuerdo con su naturaleza documental, técnica o bibliográfica. Asimismo, declaro que el contenido aquí presentado no infringe las leyes vigentes en materia de propiedad intelectual.

Albacete, a ... de ... de 2026

Fdo.: Alonso Marcos Muñoz

Resumen

Este Trabajo Fin de Máster diseña, implementa y valida una Plataforma de Gobierno del Dato centrada en la gestión de metadatos técnicos y funcionales sobre OpenMetadata y en su exportación interoperable conforme al perfil DCAT-AP-ES. Partiendo del enunciado oficial, orientado a alinear OpenMetadata con DCAT-AP, el trabajo concreta el problema en un caso de uso de validación reproducible que recorre el ciclo completo de gobierno de metadatos: desde la conexión de un sistema fuente real hasta la obtención de un catálogo interoperable validado de forma automatizada.

La plataforma se construye como un flujo extremo a extremo y reproducible. Sobre Kubernetes se despliegan OpenMetadata y un PostgreSQL de referencia con esquemas *bronze*, *silver* y *gold* que actúa como fuente técnica canónica. Sus activos se ingieren en OpenMetadata, donde se enriquecen con metadatos de gobierno, y la capa *gold* se somete a curación funcional mediante una hoja CSV versionada. Un paquete Python propio, *om_dcat_sync*, concentra toda la lógica de negocio (descubrimiento de activos, sincronización idempotente del gobierno, exportación del catálogo a JSON-LD y validación), de modo que tanto una interfaz de línea de comandos como una consola web operativa reutilizan el mismo núcleo sin duplicar reglas. La consola web permite ejecutar el flujo, editar la hoja funcional y consultar logs y artefactos sin memorizar comandos.

La validación combina pruebas automatizadas, comprobación estructural del estado vivo de OpenMetadata frente al contrato esperado y validación semántica SHACL del perfil DCAT-AP-ES para el caso de datos de alto valor (HVD), empleando shapes oficiales congeladas en el repositorio. El resultado es una plataforma simple, idempotente y reproducible que transforma activos técnicos gobernados en un catálogo RDF serializado en JSON-LD, interoperable y preparado para su federación con ecosistemas compatibles como *datos.gob.es* o *data.europa.eu*, y defendible a partir de evidencias trazables.

Abstract

This Master's Thesis designs, implements and validates a Data Governance Platform focused on managing technical and functional metadata in OpenMetadata and on its interoperable export according to the DCAT-AP-ES profile. Starting from the official topic, aimed at aligning OpenMetadata with DCAT-AP, the work frames the problem as a reproducible validation use case that spans the full metadata-governance cycle: from connecting a real source system to obtaining an interoperable catalogue validated in an automated way.

The platform is built as a reproducible end-to-end flow. OpenMetadata and a reference PostgreSQL database with `bronze`, `silver` and `gold` schemas, acting as the canonical technical source, are deployed on Kubernetes. Its assets are ingested into OpenMetadata and enriched with governance metadata, and the `gold` layer undergoes functional curation through a versioned CSV sheet. A custom Python package, `om_dcat_sync`, centralises all business logic (asset discovery, idempotent governance synchronization, JSON-LD catalogue export and validation), so that both a command-line interface and an operational web console reuse the same core without duplicating rules. The web console makes it possible to run the flow, edit the functional sheet and inspect logs and artefacts without memorising commands.

Validation combines automated tests, structural checking of OpenMetadata's live state against the expected contract, and SHACL semantic validation of the DCAT-AP-ES profile for the High-Value Datasets (HVD) case, using official shapes frozen in the repository. The result is a simple, idempotent and reproducible platform that turns governed technical assets into an interoperable RDF catalogue serialized in JSON-LD, ready for federation with compatible ecosystems such as `datos.gob.es` or `data.europa.eu`, and defensible through traceable evidence.

Agradecimientos

Quiero expresar mi agradecimiento a las personas e instituciones que han hecho posible este trabajo.

A mi tutor, Fernando Gualo Cejudo, y a mi codirector, Antonio Labian Moya, por el acompañamiento técnico y académico a lo largo de todo el desarrollo. Su orientación constante, su criterio exigente sobre el alcance, la evidencia y la trazabilidad, y su disponibilidad para revisar y discutir cada decisión han sido determinantes para dar forma y rigor a esta memoria. Les agradezco también el marco académico proporcionado por la Universidad de Castilla-La Mancha y por el Máster Universitario en Big Data y Computación en la Nube, en el que se enmarca este Trabajo Fin de Máster.

A mi familia y a las personas cercanas, por su paciencia, comprensión y apoyo incondicional durante los meses dedicados al máster y, de manera especial, a la preparación de este trabajo. Su ánimo en los momentos de mayor carga y su confianza constante han sido un sostén imprescindible para llegar hasta aquí.

Índice general

1	Introducción	1
1.1	Contexto	1
1.2	Problema abordado	2
1.3	Solución propuesta	2
1.4	Aportación del trabajo	3
1.5	Estructura del documento	4
2	Objetivos, alcance y trazabilidad	5
2.1	Objetivo general	5
2.2	Objetivos específicos	5
2.3	Alcance	6
2.4	Exclusiones	6
2.5	Cambio de alcance respecto a CKAN	7
2.6	Trazabilidad entre objetivos y evidencias	7
2.7	Metodología de trabajo	8
2.7.1	<i>Enfoque general</i>	8
2.7.2	<i>Tablero y vistas en GitHub Project</i>	9
2.7.3	<i>Fases del roadmap</i>	9
2.7.4	<i>Vista temporal del trabajo</i>	9
3	Marco técnico y estado del arte	13
3.1	Gobierno del dato y gestión de metadatos	13
3.2	Catálogos de datos y metadatos técnicos	14

3.3	RDF, JSON-LD y vocabularios semánticos	15
3.3.1	<i>Qué es RDF y por qué se utiliza.</i>	15
3.3.2	<i>JSON-LD como serialización elegida</i>	16
3.4	DCAT, DCAT-AP y DCAT-AP-ES	17
3.5	Datos de alto valor y caso HVD	18
3.6	Validación SHACL	18
3.7	OpenMetadata	20
3.7.1	<i>Gobierno centralizado frente a edición dataset por dataset</i>	20
3.8	Mercado de herramientas de gobierno alineadas con DCAT-AP-ES	21
3.8.1	<i>Posicionamiento de la Plataforma de Gobierno del Dato del TFM.</i>	22
3.9	PostgreSQL como fuente técnica reproducible	22
3.10	Docker, Kind, Kubernetes y Helm	23
3.11	Consolas web operativas y experiencia de usuario	24
3.12	Reproducibilidad, idempotencia y <i>Infrastructure as Code</i>	25
3.13	Validación frente a verificación	25
3.14	Síntesis del estado del arte	26
4	Requisitos y arquitectura de la solución	27
4.1	Requisitos funcionales	27
4.2	Requisitos no funcionales	28
4.3	No objetivos	29
4.4	Arquitectura lógica	29
4.5	Arquitectura física	32
4.6	Flujo extremo a extremo	34
4.7	Decisiones de diseño	36
4.7.1	<i>PostgreSQL como fuente canónica</i>	36
4.7.2	<i>Gobierno restringido a la capa <i>go</i>ld</i>	36
4.7.3	<i>Doble ingesta como validación multi-fuente</i>	37
4.7.4	<i>Hoja funcional para gobierno editorial</i>	37
4.7.5	<i>Configuración global por YAML</i>	39
4.7.6	<i>Núcleo único de lógica</i>	39
4.8	Modelo activo DCAT-AP-ES	40
5	Implementación	43
5.1	Organización del repositorio	43

5.2	Fuente técnica reproducible: PostgreSQL en tres capas	44
5.3	Despliegue reproducible en Kubernetes	45
5.4	Doble ingesta técnica	47
5.5	Núcleo Python: paquete <code>om_dcat_sync</code>	47
5.5.1	Configuración inmutable y <code>entrypoint</code>	49
5.5.2	Sincronización idempotente de gobierno	50
5.5.3	Exportador DCAT-AP-ES	51
5.5.4	Validación SHACL del catálogo exportado	51
5.6	Hoja funcional de gobierno y defaults globales.	51
5.7	Consola web operativa Next.js.	52
5.8	Resumen de implementación	54
6	Validación y resultados	57
6.1	Estrategia de validación	57
6.2	Pruebas automatizadas	57
6.3	Validación estructural del estado vivo	58
6.4	Validación SHACL	58
6.5	Validación externa con el validador europeo (SEMIC/ITB)	59
6.6	Resultados del caso de uso de validación	61
6.7	Artefactos generados	61
7	Discusión	63
7.1	Cumplimiento del problema planteado.	63
7.2	Beneficios obtenidos	63
7.3	Limitaciones	64
7.4	Amenazas a la validez	65
7.5	Posicionamiento frente a entornos Big Data	65
7.6	Gobierno supervisado y responsabilidad del operador.	66
7.7	Trabajo futuro	67
8	Conclusiones	69
A	Reproducción del caso de uso de validación	71
A.1	Preparación del entorno	71
A.2	Despliegue de infraestructura	71

A.3	Ejecución del workflow canónico	72
A.4	Suite reproducible	72
B	Detalle de infraestructura	73
B.1	Versión de OpenMetadata	73
B.2	Matriz de versiones del sistema	73
B.3	Valores Helm de OpenMetadata	74
B.4	Manifiesto de PostgreSQL de referencia	74
B.5	Diagrama de despliegue	75
B.6	Notas operativas	75
B.7	Instalación portable y organismos con OpenMetadata propio	75
C	Esquema de gobierno funcional	77
C.1	Estructura de la hoja <code>gold_governance.csv</code>	77
C.2	Defaults globales	77
D	Mapeo DCAT-AP-ES y entregables	79
D.1	Correspondencia OpenMetadata → DCAT-AP-ES	79
D.2	Qué gobierna OpenMetadata y qué deriva el sistema	80
D.3	Qué aporta activar el caso HVD	80
D.4	Entregables del caso de uso	80
E	Validación y consola web	81
E.1	Comandos de validación	81
E.2	Estructura de la consola web	81
E.3	Pruebas de la consola web	81
F	Demostración guiada de la plataforma	83
F.1	Propósito y supuestos iniciales	83
F.2	Apertura de la consola	84
F.3	Estado de la infraestructura	84
F.4	Servicios y datasets ingeridos	85
F.5	Curación funcional en la pantalla <i>Gobierno</i>	86
F.6	Aplicación del workflow	87
F.7	Exportación DCAT-AP-ES	88

F.8 Validación SHACL y suite reproducible	88
F.9 Artefactos y descarga del catálogo RDF	89
F.10 Historial de ejecuciones	90
F.11 Federación del catálogo	91
F.12 Resumen del recorrido	92
Referencia bibliográfica	96

Índice de figuras

1.1	Flujo lógico de la Plataforma de Gobierno del Dato. Elaboración propia. . .	3
2.1	Diagrama de Gantt del roadmap del TFM, derivado del archivo declarativo <code>scripts/planning/github_project_planificacion.json</code> . Elaboración propia.	10
2.2	Vista Kanban del GitHub Project del TFM al cierre de la fase 05. Elaboración propia.	11
3.1	Despliegue Kubernetes reproducible usado como base del caso de uso de validación. Elaboración propia.	24
4.1	Arquitectura lógica por capas y responsabilidades: fuente técnica, catálogo técnico, gobierno funcional, núcleo Python y operación. Elaboración propia.	31
4.2	Arquitectura física de despliegue: host con Docker y clúster Kind que ejecuta OpenMetadata, sus dependencias y el PostgreSQL de referencia. Elaboración propia.	33
4.3	Flujo operativo del caso de uso de validación. Elaboración propia.	34
4.4	Pipeline de metadatos desde OpenMetadata hasta validación SHACL. Elaboración propia.	35
4.5	Mapeo activo entre clases DCAT-AP-ES y elementos de la plataforma. Elaboración propia.	42
5.1	Árbol canónico del repositorio del TFM. Elaboración propia.	44
5.2	Esquemas bronze, silver y gold del PostgreSQL de referencia, vistos desde la User Interface (UI) de OpenMetadata (OM). Elaboración propia. . . .	45
5.3	Contenedores Docker que materializan el clúster Kind y los nodos auxiliares. Elaboración propia.	46
5.4	Pods desplegados en el clúster Kind: OpenMetadata, MySQL, OpenSearch y PostgreSQL de referencia. Elaboración propia.	46
5.5	Servicios de base de datos ingeridos en OpenMetadata tras la doble ingesta. Elaboración propia.	47
5.6	Estructura interna del paquete <code>om_dcat_sync</code> . Elaboración propia.	48

5.7	Propiedades personalizadas DCAT y etiquetas <code>dcat_theme.*</code> aplicadas a una tabla <code>gold</code> en OpenMetadata tras ejecutar el workflow. Elaboración propia.	50
5.8	Pantalla <i>Gobierno</i> de la consola web: edición de la hoja <code>gold_governance.csv</code> . Elaboración propia.	52
5.9	Pantalla <i>Workflow</i> : lanzamiento de <code>dry-run</code> y <code>apply</code> con resumen de cambios. Elaboración propia.	53
5.10	Pantalla <i>DCAT</i> : exportación y previsualización del catálogo JavaScript Object Notation for Linked Data (JSON-LD). Elaboración propia.	53
5.11	Pantalla <i>Validación</i> : ejecución de la suite reproducible y resumen de resultados. Elaboración propia.	54
6.1	Validación externa del catálogo en el <i>SEMIC SHACL Validator</i> (ITB, Comisión Europea): perfil DCAT-AP HVD 3.0.0 Base Zero, resultado SUCCESS sin errores ni advertencias. Elaboración propia.	60
6.2	Informe de validación HTML renderizado en el navegador desde la consola web: resumen de conformidad por indicador y detalle del resultado. Elaboración propia.	62
F.1	Pantalla inicial de la consola web y <i>sidebar</i> con las secciones operativas. Elaboración propia.	84
F.2	Pantalla <i>Infraestructura</i> : comprobación de prerequisites y opciones de reset limpio. Elaboración propia.	85
F.3	Pantalla <i>Ingesta</i> : servicios PostgreSQL y datasets <code>gold</code> descubiertos por OpenMetadata. Elaboración propia.	85
F.4	Pantalla <i>Gobierno</i> : edición de la hoja <code>gold_governance.csv</code> con listas controladas para temática y categoría High Value Datasets (Conjuntos de Datos de Alto Valor) (HVD). Elaboración propia.	86
F.5	Pantalla <i>Workflow</i> : <code>dry-run</code> y aplicación con resumen de cambios. Elaboración propia.	87
F.6	Pantalla <i>DCAT</i> : exportación del catálogo JSON-LD y previsualización. Elaboración propia.	88
F.7	Pantalla <i>Validación</i> : ejecución de la suite reproducible y resumen consolidado. Elaboración propia.	89
F.8	Pantalla <i>SHACL</i> : manifiesto de las <i>shapes</i> DCAT-AP-ES congeladas (sin descarga en tiempo de ejecución). Elaboración propia.	89
F.9	Pantalla <i>Artefactos</i> : listado de evidencias con vista previa (<i>Ver</i>) y apertura nativa (<i>Abrir</i>) para HTML renderizado y PDF en el visor del navegador. Elaboración propia.	90
F.10	Pantalla <i>Ejecuciones</i> : historial de <i>jobs</i> con estado, duración, código de salida y acceso al log y artefactos. Elaboración propia.	91

Índice de tablas

2.1	Trazabilidad entre objetivos, capítulos y evidencias. Elaboración propia. . .	8
2.2	Fases del roadmap del TFM con su área DAMA-DMBOK predominante. Elaboración propia, sincronizada con <code>scripts/planning/github_project_planificacion.json</code>	9
4.1	Requisitos funcionales con criterio de aceptación. Elaboración propia. . . .	28
4.2	Requisitos no funcionales y compromisos asociados. Elaboración propia. . .	29
4.3	Columnas de la hoja funcional <code>gold_governance.csv</code> . Elaboración propia.	38
5.1	Bloques principales del repositorio. Elaboración propia.	43
5.2	Módulos principales de <code>tfm_ingestor</code> . Elaboración propia.	49
6.1	Resumen de resultados del caso de uso de validación. Elaboración propia a partir de la documentación del repositorio.	61
B.1	Matriz de versiones fijadas del sistema. Elaboración propia.	74
D.1	Correspondencia mínima OpenMetadata → DCAT-AP-ES. Elaboración propia a partir de <code>docs/dcat_mapping.md</code>	79
D.2	Entregables del caso de uso de validación y su visualización en la memoria. Elaboración propia.	80
F.1	Mapa del recorrido demo: acción del operario, artefacto producido y sección de la memoria donde se justifica formalmente. Elaboración propia.	92

Índice de Códigos

3.1	Tripletas RDF que describen un dataset DCAT en sintaxis Turtle.	16
3.2	Mismo grafo serializado en JSON-LD usando @context para acortar las IRIs.	16
3.3	Fragmento ilustrativo de SHACL, serializado en Turtle, que exige título y descripción a cada Dataset DCAT.	19
4.1	Perfil operativo <code>operational_profile.yaml</code> que enlaza defaults, reglas, hoja y caso de validación.	39
4.2	Configuración inmutable del workflow canónico en <code>workflow_service.py</code>	40
5.1	Fragmento de <code>sql/opendata_demo_init.sql</code> : tabla publicable de la capa gold.	44
5.2	Fragmento de <code>k8s/openmetadata.values.yaml</code> : configuración mínima del release Helm.	45
5.3	Esqueleto de <code>run_workflow</code> en <code>workflow_service.py</code>	49
5.4	Contexto JSON-LD canónico generado por <code>dcat_export.py</code>	51
5.5	Estructura de un <i>job</i> persistido en <code>state/web_jobs/</code>	54
6.1	Ejecución de la suite <code>pytest</code> del núcleo Python y de los tests <code>vitest</code> de la consola web.	58
6.2	Validación estructural del estado vivo de OpenMetadata desde el CLI <code>om_dcat_sync</code>	58
6.3	Validación SHACL aislada del catálogo JSON-LD ya exportado, caso HVD.	58
6.4	Workflow canónico end-to-end: sincroniza gobierno, exporta DCAT-AP-ES y valida SHACL HVD.	59
A.1	Instalación de dependencias Python del núcleo <code>om_dcat_sync</code>	71
A.2	Instalación de dependencias de la consola web con <code>pnpm</code>	71
A.3	Lanzamiento de Kind, Helm e ingesta doble.	71
A.4	Ejecución del workflow canónico end-to-end.	72
A.5	Suite reproducible documentada.	72
E.1	Validación independiente y por suite.	81
E.2	Pruebas Vitest de la consola web.	81
F.1	Comprobación rápida del entorno antes de iniciar la demostración.	83
F.2	Envío del catálogo a un harvester CKAN con extensión DCAT.	91

Índice de Acrónimos

DAMA	Data Management Association International
DMBOK	Data Management Body of Knowledge
HVD	High Value Datasets (Conjuntos de Datos de Alto Valor)
FQN	Fully Qualified Name (identificador completo de activo en OpenMetadata)
CSV	Comma-Separated Values
RDF	Resource Description Framework
RDFS	Resource Description Framework Schema
OWL	Web Ontology Language
SPARQL	SPARQL Protocol and RDF Query Language
IRI	Internationalized Resource Identifier
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
JSON-LD	JavaScript Object Notation for Linked Data
XML	eXtensible Markup Language
TTL	Turtle (Terse RDF Triple Language)
DCAT	Data Catalog Vocabulary (W3C)
DCAT-AP	DCAT Application Profile for European Data Portals
DCAT-AP-ES	Perfil de aplicación DCAT-AP para España
SKOS	Simple Knowledge Organization System
NTI-RISP	Norma Técnica de Interoperabilidad de Reutilización de Recursos de Información del Sector Público
SHACL	Shapes Constraint Language
W3C	World Wide Web Consortium

API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
SQL	Structured Query Language
YAML	YAML Ain't Markup Language
CI/CD	Continuous Integration / Continuous Delivery
LDAP	Lightweight Directory Access Protocol
RBAC	Role-Based Access Control
SSO	Single Sign-On
OM	OpenMetadata
CLI	Command Line Interface
UI	User Interface
PMBOK	Project Management Body of Knowledge
UCLM	Universidad de Castilla-La Mancha

1. Introducción

1.1. Contexto

El Trabajo Fin de Máster se enmarca en el diseño y despliegue de una configuración de metadatos en OpenMetadata alineada con DCAT-AP, con el objetivo de mejorar la interoperabilidad de catálogos de datos en entornos Big Data y de computación en la nube. La ficha oficial del TFE define explícitamente el análisis de DCAT-AP, su correspondencia con OpenMetadata, la configuración de taxonomías y metadatos personalizados, y la validación mediante exportación en RDF, JSON-LD o formato equivalente [1].

El trabajo se concreta en una **Plataforma de Gobierno del Dato**. La plataforma no procesa datos de negocio con finalidad analítica, sino que gobierna metadatos: identifica activos técnicos, completa metadatos funcionales, genera un catálogo interoperable y valida su conformidad. Esta distinción es importante porque el valor técnico del proyecto no reside en calcular métricas sobre las tablas de ejemplo, sino en demostrar un ciclo reproducible de gobierno e interoperabilidad de metadatos.

En el ámbito de los datos abiertos, DCAT permite describir catálogos, conjuntos de datos, distribuciones y servicios de datos mediante RDF, facilitando el intercambio de metadatos entre sistemas [2]. En España, DCAT-AP-ES concreta este enfoque para el contexto nacional, tomando como referencia DCAT-AP 2.1.1 e incorporando elementos de DCAT-AP HVD 2.2.0 para conjuntos de datos de alto valor [3]. Esta evolución justifica que el trabajo no se limite a crear campos personalizados en OpenMetadata, sino que produzca una salida interoperable y validable.

1.2. Problema abordado

OpenMetadata proporciona capacidades de catálogo, descubrimiento y gobierno de metadatos, pero no resuelve de forma nativa, en el repositorio de este TFM, la generación de un catálogo DCAT-AP-ES completo y validado a partir de activos técnicos gobernados. El problema consiste en cerrar esa separación entre dos mundos:

- por un lado, el catálogo técnico gestionado en OpenMetadata, con servicios, bases de datos, esquemas, tablas, columnas, descripciones, etiquetas y metadatos personalizados;
- por otro lado, el modelo DCAT-AP-ES, que espera un grafo RDF con clases y propiedades específicas para catálogos, datasets, distribuciones, servicios de datos y agentes.

La dificultad no es únicamente transformar nombres de campos. También hay que decidir qué activos se consideran publicables, qué metadatos deben ser curados por una persona responsable del catálogo, qué valores pueden derivarse por configuración y cómo validar que la salida cumple un perfil semántico externo. Por ello, el trabajo combina infraestructura, integración con API, modelado de metadatos, exportación semántica, validación SHACL y una consola web operativa.

1.3. Solución propuesta

La solución implementada parte de un PostgreSQL de referencia desplegado junto a OpenMetadata. Ese PostgreSQL contiene tres capas, *bronze*, *silver* y *gold*. Para el caso de uso de validación se realiza una doble ingesta de la misma fuente técnica bajo dos servicios de OpenMetadata: `postgres_demo_service` y `postgres_validation_service`. Esta decisión permite mostrar gobierno multi-fuente sin introducir una segunda base de datos artificial ni aumentar innecesariamente la complejidad de infraestructura.

OpenMetadata ingiere los activos técnicos de ambos servicios y la plataforma gobierna únicamente las tablas *gold*, que representan los datasets publicables. Las capas *bronze* y *silver* permanecen visibles en el catálogo técnico pero quedan fuera del catálogo de publicación; la justificación detallada de esta frontera, alineada con la arquitectura *medallion* y con el marco DAMA, se desarrolla en la sección 4.7.2. Como existen dos servicios y dos tablas *gold* por servicio, el catálogo exportado contiene cuatro datasets publicables. La hoja `gold_governance.csv` identifica cada fila por su `table_fqn`, de modo que dos tablas con el mismo nombre funcional no se mezclan.

El núcleo de la solución es el paquete Python `om_dcat_sync`, conservado internamente como `tfm_ingestor`. Este componente descubre activos técnicos en OpenMetadata, refresca una hoja funcional de gobierno, aplica metadatos de forma idempotente, exporta un catálogo JSON-LD y valida la salida con SHACL. La app web situada en `web/` no duplica la lógica, sino que ejecuta los mismos scripts y comandos versionados desde una interfaz operativa.

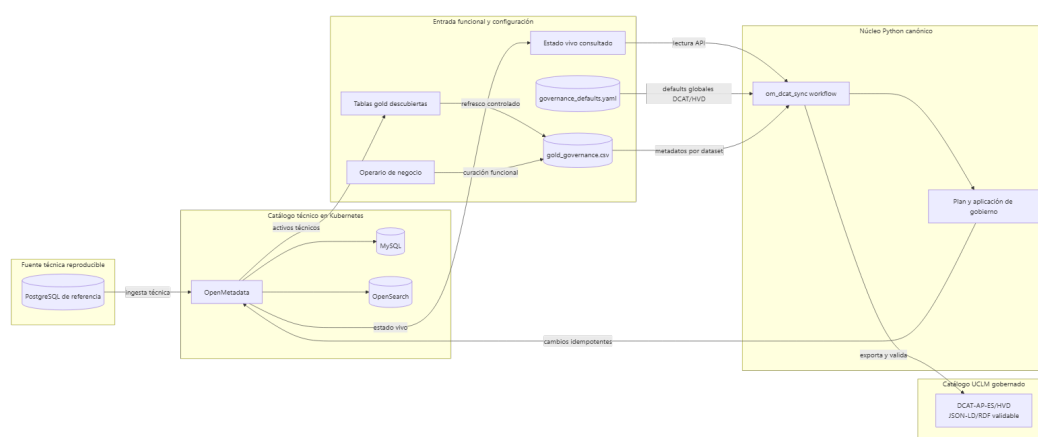


Figura 1.1: Flujo lógico de la Plataforma de Gobierno del Dato. Elaboración propia.

1.4. Aportación del trabajo

La aportación principal es una capa reproducible de gobierno e interoperabilidad construida sobre OpenMetadata. El trabajo aporta:

- un despliegue reproducible de OpenMetadata y PostgreSQL mediante Kubernetes, Helm y scripts versionados;
- una doble ingesta PostgreSQL que permite validar gobierno multi-fuente con activos técnicos reales;
- un modelo operativo que separa activos técnicos ingeridos de datasets funcionalmente publicables;
- una hoja de gobierno para curar título, descripción, publicador, temática, categoría HVD y URL de acceso por dataset;

-
- un exportador JSON-LD que genera las clases activas `dcat:Catalog`, `dcat:Dataset`, `dcat:Distribution`, `dcat:DataService` y `foaf:Agent`;
 - una validación reproducible mediante SHACL, con shapes DCAT-AP-ES locales y congeladas;
 - una consola web que permite ejecutar el caso de uso de validación, revisar logs y consultar artefactos sin introducir comandos arbitrarios.

1.5. Estructura del documento

El capítulo 2 define objetivos, alcance y trazabilidad. El capítulo 3 presenta el marco técnico y el estado del arte antes de describir la implementación propia. El capítulo 4 concreta requisitos, decisiones de diseño y arquitectura de la plataforma. El capítulo 5 detalla la implementación. El capítulo 6 recoge la metodología de validación y los resultados. El capítulo 7 discute beneficios, limitaciones y amenazas a la validez. El capítulo 8 presenta las conclusiones.

2. Objetivos, alcance y trazabilidad

2.1. Objetivo general

El objetivo general del TFM es diseñar y desplegar una configuración de metadatos en OpenMetadata alineada con DCAT-AP, concretada en este repositorio como DCAT-AP-ES, para mejorar la interoperabilidad y la gestión semántica de catálogos de datos en entornos Big Data y de computación en la nube [1].

En términos implementados, este objetivo se materializa en una Plataforma de Gobierno del Dato capaz de descubrir activos técnicos, enriquecerlos con metadatos funcionales, exportar un catálogo JSON-LD y validar su conformidad mediante SHACL. La validación se apoya en dos servicios PostgreSQL ingeridos en OpenMetadata, lo que permite demostrar que el gobierno no depende de una única fuente ni de nombres de tabla globalmente únicos.

2.2. Objetivos específicos

Los objetivos específicos se formulan de forma verificable:

1. Analizar DCAT-AP y su concreción DCAT-AP-ES, incluyendo clases, propiedades, vocabularios controlados y validación SHACL.
2. Mapear las entidades de OpenMetadata con las clases principales de DCAT-AP-ES.
3. Configurar en OpenMetadata los metadatos mínimos necesarios para representar datasets publicables.
4. Implementar un workflow reproducible de sincronización de metadatos desde activos técnicos reales.

-
5. Ejecutar doble ingesta PostgreSQL para validar gobierno multi-fuente con tablas homónimas.
 6. Exportar el catálogo gobernado en JSON-LD compatible con DCAT-AP-ES.
 7. Validar la salida con SHACL y evaluar beneficios, limitaciones, idempotencia y mantenibilidad.
 8. Proporcionar una consola web operativa que ejecute el mismo flujo sin duplicar reglas de negocio.

2.3. Alcance

El alcance del trabajo queda delimitado por el caso de uso de validación. La plataforma gobierna metadatos generados a partir de un PostgreSQL de referencia, no datos de negocio reales. Las tablas se organizan en capas *bronze*, *silver* y *gold*; solo *gold* se considera publicable.

La implementación activa cubre:

- despliegue local reproducible con Kubernetes, Helm y scripts PowerShell;
- doble ingesta técnica de PostgreSQL en OpenMetadata bajo los servicios `postgres_demo_service` y `postgres_validation_service`;
- gobierno funcional de las tablas *gold* de ambos servicios, con cuatro datasets publicables;
- custom properties y etiquetas DCAT en OpenMetadata;
- exportación JSON-LD;
- validación SHACL con el caso HVD;
- pruebas automatizadas y validación estructural;
- consola web operativa basada en Next.js.

2.4. Exclusiones

Quedan fuera del alcance:

- publicación automática en un portal externo CKAN o `datos.gob.es`;

- certificación jurídica de datasets de alto valor fuera del caso de uso de validación;
- calidad fila a fila de los datos de negocio;
- alta disponibilidad, SSO, RBAC avanzado, cifrado avanzado, backups productivos y observabilidad avanzada;
- flujos editoriales complejos propios de plataformas comerciales completas.

2.5. Cambio de alcance respecto a CKAN

La ficha oficial proponía implementar un pipeline de ingesta o sincronización de metadatos desde una fuente externa, por ejemplo CKAN [1]. Durante el desarrollo se descartó CKAN como origen operativo principal. La razón es metodológica: CKAN actúa principalmente como catálogo de publicación e intercambio de metadatos ya elaborados, mientras que el objetivo académico de este TFM requiere demostrar cómo se generan, gobiernan y validan metadatos desde activos técnicos reproducibles.

Desde el punto de vista de gobierno del dato, el enfoque DAMA enfatiza control, roles, metadatos, calidad e interoperabilidad como funciones coordinadas de gestión [4, 5]. Por ello, partir de PostgreSQL y OpenMetadata permite evidenciar mejor el ciclo completo: descubrimiento técnico, curación funcional, aplicación de gobierno, exportación y validación. CKAN se mantiene como posible destino o punto de federación futura, no como fuente canónica del caso de uso de validación.

2.6. Trazabilidad entre objetivos y evidencias

Objetivo	Capítulo	Evidencia del repositorio
Análisis DCAT-AP-ES	Marco técnico	docs/dcat_mapping.md, bibliografía oficial
Mapeo OpenMetadata	Diseño e implementación	dcat_export.py, governance_service.py
Configuración de gobierno	Implementación	gold_governance.csv, governance_defaults.yaml
Doble ingesta PostgreSQL	Implementación y validación	ingest_postgres_double.ps1, gold_governance.csv
Workflow reproducible	Implementación y validación	workflow_service.py, run_validation_suite.ps1
Exportación JSON-LD	Validación	dcat_export.py, artefactos *.jsonld
Validación SHACL	Validación	shacl_validation.py, shapes vendorizadas
Consola operativa	Implementación	web/, operations.ts, jobRunner.ts

Tabla 2.1: Trazabilidad entre objetivos, capítulos y evidencias. Elaboración propia.

2.7. Metodología de trabajo

2.7.1. Enfoque general

El TFM se ha gestionado con un enfoque *lean* basado en **Kanban** [6], complementado con los procesos de planificación, seguimiento y cierre del Project Management Body of Knowledge (PMBOK) [7] adaptados a un trabajo individual. La elección no es ornamental: Kanban encaja con un trabajo de alcance acotado y entrega incremental, mientras que el PMBOK aporta vocabulario y disciplina para los hitos académicos (planificación, ejecución, validación, cierre). Esta combinación se alinea con las prácticas habituales del sector cuando un único responsable cubre los roles de analista, desarrollador y revisor.

Las reglas operativas se han fijado de manera explícita para evitar deriva:

- **Límite de trabajo en curso (WIP):** máximo 2-3 tareas simultáneas, una sola en En curso cuando incluye despliegue o cambios de infraestructura.
- **Revisión semanal:** cierre y replanificación cada lunes; revisión adicional al cierre de cada fase.
- **Definición de Hecho (DoD):** una tarea solo se mueve a Hecho cuando existe evidencia versionada en el repositorio (código, configuración, documentación o artefacto regenerable) y, en su caso, tests verdes.

2.7.2. Tablero y vistas en GitHub Project

La herramienta canónica de seguimiento es **GitHub Projects v2** [8], sincronizada con los *issues* del propio repositorio. El tablero combina seis estados (Backlog, Por definir, En curso, Bloqueado, En revisión, Hecho) con campos personalizados de fase, tipo de tarea, fecha de inicio y fecha de fin. Esta estructura permite obtener simultáneamente una vista Kanban, un *roadmap* cronológico (Gantt) y un listado de evidencias por tarea.

La sincronización no es manual: el archivo declarativo `scripts/planning/github_project_planificacion.json` contiene la fuente de verdad del roadmap completo. El script `scripts/planning/bootstrap_github_project.py` aplica ese archivo contra el Project remoto de forma idempotente, creando o actualizando *milestones*, *issues*, etiquetas y campos del Project sin duplicar items. Esta automatización garantiza que el tablero refleja el estado real del trabajo planificado y evita la deriva entre documentación y herramienta.

2.7.3. Fases del roadmap

El trabajo se ha organizado en seis fases secuenciales con superposición controlada. La tabla 2.2 resume cada fase con su ventana temporal real, las áreas Data Management Body of Knowledge (DMBOK) [5] implicadas y el estado al cierre del cuerpo de la memoria.

Fase	Ventana	Área DMBOK implicada
01 Planificación	feb–abr 2026	Gobierno, arquitectura
02 Modelo DCAT-AP-ES	mar–abr 2026	Metadatos, interoperabilidad
03 Configuración OpenMetadata	abr 2026	Metadatos, integración
04 Pipeline de ingesta	abr–may 2026	Integración, gobierno
05 Validación	may 2026	Calidad, gobierno
06 Memoria y defensa	may–jul 2026	Documentación

Tabla 2.2: Fases del roadmap del TFM con su área DAMA-DMBOK predominante. Elaboración propia, sincronizada con `scripts/planning/github_project_planificacion.json`.

2.7.4. Vista temporal del trabajo

La figura 2.1 muestra el diagrama de Gantt derivado del Project. Las fases se solapan donde la naturaleza del trabajo lo permitía (por ejemplo, la fase 02 de modelo DCAT-AP-ES

arrancó mientras se cerraban riesgos de la fase 01). El bloque de redacción de la memoria está reservado a las últimas semanas, con la defensa en julio de 2026 como hito final.

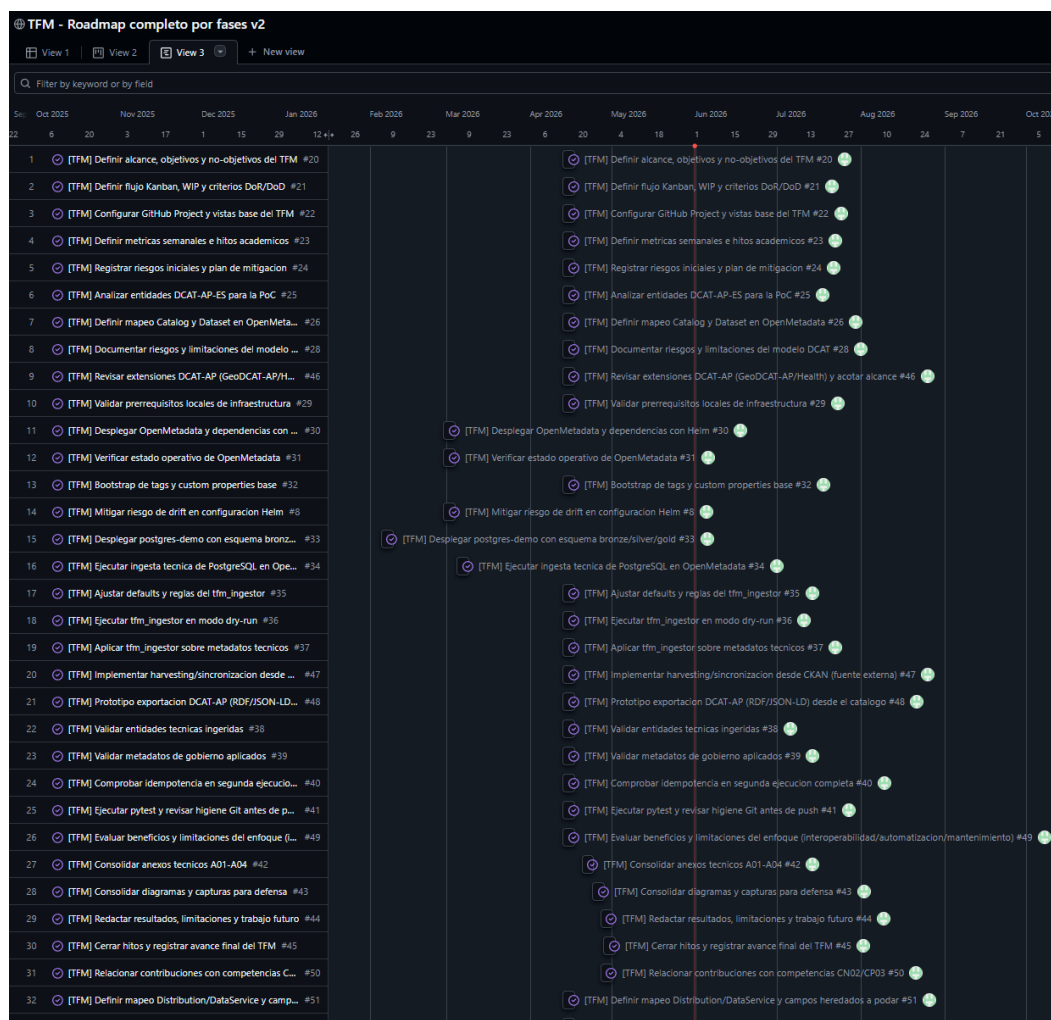


Figura 2.1: Diagrama de Gantt del roadmap del TFM, derivado del archivo declarativo `scripts/planning/github_project_planificacion.json`. Elaboración propia.

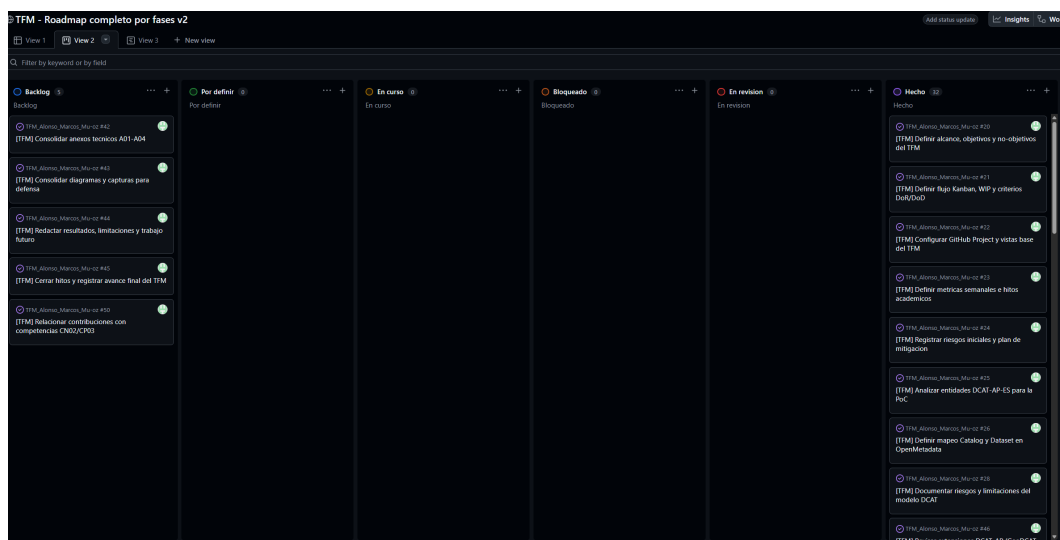


Figura 2.2: Vista Kanban del GitHub Project del TFM al cierre de la fase 05. Elaboración propia.

3. Marco técnico y estado del arte

Este capítulo explica primero los conceptos y tecnologías que forman el estado del arte utilizado por el TFM. La implementación propia se reserva para capítulos posteriores. La separación es deliberada: antes de justificar cómo se ha construido la plataforma conviene entender qué problema resuelven el gobierno del dato, los catálogos de metadatos, DCAT-AP-ES, SHACL, OpenMetadata, Kubernetes y la interfaz web en el contexto actual.

3.1. Gobierno del dato y gestión de metadatos

El gobierno del dato es el conjunto de responsabilidades, políticas, procesos, roles y controles que permite gestionar los datos como activos de una organización. En la práctica actual no se limita a almacenar información: define quién es responsable de cada activo, qué significado tiene, bajo qué reglas se publica, qué calidad mínima debe tener, cómo se traza su origen y cómo se comparte con terceros. Data Management Association International (DAMA)-DMBOK se utiliza habitualmente como marco de referencia para ordenar estas funciones de gestión de datos [5]. En el ámbito español de datos abiertos, datos.gob.es relaciona estas prácticas con principios como accesibilidad, consistencia, auditabilidad, exactitud y seguridad [4].

DMBOK agrupa once áreas funcionales: arquitectura de datos, modelado y diseño, almacenamiento y operaciones, seguridad, integración e interoperabilidad, documentación y contenido, datos maestros y de referencia, almacenamiento analítico y *business intelligence*, metadatos, calidad y gobierno [5]. La Plataforma del TFM se sitúa de forma intencionada en la intersección de tres áreas: **metadatos** (descripción semántica de activos), **integración e interoperabilidad** (exportación a un perfil externo) y **gobierno** (políticas sobre qué se publica y cómo). El resto de áreas no se aborda porque corresponden a programas de gestión de

datos de mayor envergadura, con equipos dedicados y objetivos transversales.

La gestión de metadatos es una pieza central de ese gobierno. Los metadatos describen los datos y permiten que una persona o un sistema entiendan qué existe, dónde se encuentra, quién lo mantiene, qué reglas le aplican y cómo puede reutilizarse. En entornos modernos, los metadatos ya no se mantienen solo en documentos estáticos: se capturan desde bases de datos, almacenes analíticos, herramientas de integración, sistemas de calidad, catálogos y plataformas de linaje. El objetivo es reducir la distancia entre el sistema técnico que contiene el activo y la descripción funcional que se publica o consulta.

DMBOK distingue cuatro tipos de metadatos: *de negocio* (significado funcional, glosario, responsable), *técnicos* (esquema, tipo, longitud, claves), *operacionales* (frecuencia de actualización, latencia, propietario de la ejecución) y *de gobierno* (clasificación, sensibilidad, calidad esperada) [5]. La Plataforma del TFM gobierna principalmente metadatos técnicos descubiertos por OM y metadatos de negocio aportados por la hoja funcional, dejando el resto fuera del alcance porque no son esenciales para demostrar conformidad DCAT-AP-ES.

La Plataforma de Gobierno del Dato del TFM adopta esa visión y la acota deliberadamente a un perímetro reproducible. No persigue medir la calidad de cada valor almacenado en tablas ni sustituir una suite corporativa completa, decisiones que pertenecen a iniciativas con alcance, presupuesto y ciclo de vida distintos. Su objetivo es entregar un ciclo defendible y verificable: descubrir activos técnicos, enriquecerlos con metadatos funcionales, publicar una representación interoperable y validar formalmente que esa representación cumple un perfil externo.

3.2. Catálogos de datos y metadatos técnicos

Un catálogo de datos organiza descripciones de activos para que puedan encontrarse, entenderse y gobernarse. En el estado actual de la ingeniería de datos, los catálogos suelen integrar conectores de ingesta, modelos de entidades, búsqueda, etiquetas, glosarios, owners, descripciones, linaje y APIs. La utilidad principal es que los equipos de datos dejen de depender de conocimiento informal y puedan consultar un inventario vivo.

La diferencia entre datos y metadatos es clave. Una tabla PostgreSQL contiene filas y columnas con datos. OpenMetadata registra que existe esa tabla, su nombre, esquema, columnas, descripción, etiquetas y propiedades adicionales. DCAT-AP-ES, por su parte, no espera la tabla como tal, sino una descripción semántica del dataset publicable. Por eso el problema de este TFM no es mover datos de negocio, sino transformar metadatos técnicos y funcionales en un grafo de catálogo.

También es importante distinguir catálogo técnico de catálogo de publicación. Un catálogo técnico ayuda a operar internamente los activos. Un catálogo de publicación, como los usados para datos abiertos, exige metadatos comprensibles, vocabularios controlados y formatos interoperables. La Plataforma de Gobierno del Dato une ambos niveles: parte de activos técnicos en OpenMetadata y produce una salida DCAT-AP-ES validable.

3.3. RDF, JSON-LD y vocabularios semánticos

3.3.1. Qué es RDF y por qué se utiliza

Resource Description Framework (RDF) (*Resource Description Framework*) es la recomendación del World Wide Web Consortium (W3C) para representar información en la Web como un grafo dirigido y etiquetado [9]. La unidad mínima de RDF es la *tripleta*: un enunciado con sujeto, predicado y objeto. El sujeto identifica un recurso, el predicado nombra una relación o propiedad y el objeto puede ser otro recurso o un valor literal con tipo (cadena, fecha, número, etc.). Al encadenar muchas tripletas se obtiene un grafo en el que cualquier sistema puede navegar relaciones sin conocer el esquema interno del sistema fuente.

La ventaja decisiva de RDF respecto a otros modelos de descripción es el uso de Internationalized Resource Identifiers (IRIs)/Uniform Resource Identifiers (URIs) como identificadores estables. Si dos catálogos diferentes utilizan la misma IRI de la clase *Dataset* del vocabulario Data Catalog Vocabulary (W3C) (DCAT) (`dcat:Dataset`, expandida a la IRI oficial publicada por el W3C), ambos están hablando de la misma entidad aunque sus bases de datos internas sean distintas. Esa propiedad permite que organizaciones independientes describan datos de forma interoperable y que portales agregadores combinen catálogos heterogéneos.

RDF no impone una sintaxis concreta. Las mismas tripletas pueden serializarse en distintos formatos según el caso de uso: eXtensible Markup Language (XML)-RDF para herramientas históricas, Turtle (Turtle (Terse RDF Triple Language) (TTL)) para legibilidad humana, N-Triples para flujos línea a línea o JSON-LD para integración con tecnologías web. La elección del formato no cambia el grafo subyacente.

Sobre RDF se construyen capas adicionales habituales en gobierno de datos:

- **Resource Description Framework Schema (RDFS)**: permite declarar clases, jerarquías de clases y dominio/rango de las propiedades.
- **Web Ontology Language (OWL)**: añade lógica descriptiva para definir restricciones, equivalencias y reglas más expresivas.

-
- **Simple Knowledge Organization System (SKOS)**: estandariza vocabularios controlados, taxonomías y listas temáticas, muy utilizado para describir temas, categorías o licencias en catálogos.
 - **SPARQL Protocol and RDF Query Language (SPARQL)**: lenguaje de consulta para grafos RDF; equivalente conceptual a Structured Query Language (SQL) pero sobre tripletas.

3.3.2. JSON-LD como serialización elegida

JSON-LD es una serialización de RDF basada en JavaScript Object Notation (JSON) [10]. Conserva las tripletas, pero las expresa con la sintaxis que ya manejan los entornos web actuales. Tres elementos lo distinguen: el `@context`, que asocia nombres cortos con IRIs completas; el `@id`, que identifica unívocamente cada recurso; y el `@type`, que indica la clase RDF a la que pertenece.

El listado 3.1 muestra las mismas tripletas para un dataset DCAT en sintaxis Turtle, y el listado 3.2 las muestra en JSON-LD. Ambos describen el mismo grafo RDF.

```
1 @prefix dcat: <http://www.w3.org/ns/dcat#> .
2 @prefix dct:  <http://purl.org/dc/terms/> .
3 @prefix ex:   <https://www.uclm.es/datos/plataforma-gobierno-dato/> .
4
5 ex:agenda-cultural-publica a dcat:Dataset ;
6     dct:title      "Agenda cultural pública - servicio operativo"@es ;
7     dct:description "Relación de eventos culturales públicos con fecha,
8     ↪ municipio, categoría y asistencia estimada."@es ;
9     dct:publisher  ex:UCLM ;
10    dcat:theme      <http://datos.gob.es/kos/sector-publico/sector/cultura-ocio>
11    ↪ .
```

Código 3.1: Tripletas RDF que describen un dataset DCAT en sintaxis Turtle.

```
1 {
2   "@context": {
3     "dcat": "http://www.w3.org/ns/dcat#",
4     "dct":  "http://purl.org/dc/terms/",
5     "ex":   "https://www.uclm.es/datos/plataforma-gobierno-dato/"
6   },
7   "@id":    "ex:agenda-cultural-publica",
```

```

8  "@type": "dcat:Dataset",
9  "dct:title":      { "@value": "Agenda cultural pública - servicio operativo",
    ↪  "@language": "es" },
10 "dct:description": { "@value": "Relación de eventos culturales públicos con
    ↪  fecha, municipio, categoría y asistencia estimada.", "@language": "es" },
11 "dct:publisher":   { "@id": "ex:UCLM" },
12 "dcat:theme":      { "@id": "http://datos.gob.es/kos/sector-publico/sector/
    ↪  cultura-ocio" }
13 }

```

Código 3.2: Mismo grafo serializado en JSON-LD usando @context para acortar las IRIs.

En este TFM, JSON-LD es el formato canónico del catálogo exportado por tres razones prácticas:

1. Es legible para una persona que revisa el artefacto en el repositorio, sin necesidad de herramientas RDF especializadas.
2. Puede versionarse en Git como un archivo de texto plano y diferenciarse limpiamente entre ejecuciones.
3. Es entrada directa para validadores Shapes Constraint Language (SHACL) y para portales que aceptan DCAT-AP-ES.

La consecuencia metodológica es que la interoperabilidad real no se obtiene por usar JSON: se obtiene por usar *vocabularios y perfiles* compartidos. Por eso la memoria se apoya en DCAT, DCAT Application Profile for European Data Portals (DCAT-AP) y DCAT-AP-ES, que aportan clases, propiedades y restricciones conocidas para describir catálogos de datos.

3.4. DCAT, DCAT-AP y DCAT-AP-ES

DCAT es el vocabulario del W3C para describir catálogos de datos y facilitar el intercambio de metadatos entre sistemas [2]. Su modelo incluye clases como `dcat:Catalog`, `dcat:Dataset`, `dcat:Distribution` y `dcat:DataService`. La idea principal es separar el recurso conceptual, sus formas de acceso y los servicios que lo hacen disponible.

DCAT-AP adapta DCAT al contexto europeo de portales de datos. Su objetivo es que los catálogos del sector público puedan intercambiar descripciones de datasets de manera homogénea y facilitar agregación por portales nacionales o europeos [11]. DCAT-AP no susti-

tuye a DCAT: lo perfila, selecciona propiedades, cardinalidades y recomendaciones para un ámbito concreto.

DCAT-AP-ES concreta ese perfil para España. La documentación oficial de DCAT-AP-ES 1.0.0 indica que adopta DCAT-AP 2.1.1 e incorpora elementos de DCAT-AP HVD 2.2.0 para describir conjuntos de datos de alto valor [3]. Además, define relaciones, vocabularios controlados, convenciones y material de validación. Esta concreción es relevante para el TFM porque el enunciado oficial habla de DCAT-AP, pero la implementación operativa necesita un perfil verificable en el contexto español.

3.5. Datos de alto valor y caso HVD

La regulación europea sobre conjuntos de datos de alto valor ha aumentado la importancia de describir correctamente categorías, licencias, servicios de acceso y legislación aplicable. En DCAT-AP-ES, el caso HVD incorpora restricciones adicionales sobre datasets, distribuciones y servicios de datos [12]. Para un proyecto académico, activar HVD tiene valor porque obliga a modelar más que un título y una descripción: exige relacionar dataset, distribución, servicio, categoría, legislación, licencia y contacto.

En esta memoria la clasificación HVD se interpreta como hipótesis de diseño del caso de uso de validación. No equivale a una calificación jurídica automática de datos reales. Su papel es técnico: ejercitar el perfil más exigente y demostrar que el exportador puede construir una salida formalmente validable.

3.6. Validación SHACL

Apoyándose en los conceptos de RDF, JSON-LD y DCAT-AP-ES ya introducidos, esta sección describe el mecanismo de validación formal del catálogo. SHACL (*Shapes Constraint Language*) es un lenguaje del W3C para validar grafos RDF mediante *shapes* [13]. Una *shape* describe restricciones que deben cumplir los nodos de un grafo: clases esperadas, propiedades obligatorias, cardinalidades, tipos de valor, severidades o relaciones entre recursos. En un catálogo DCAT-AP-ES, SHACL permite comprobar si el JSON-LD exportado contiene las piezas exigidas por el perfil.

La validación SHACL no garantiza que el dataset sea útil, legalmente publicable o editorialmente perfecto. Garantiza que el grafo cumple restricciones formales. Esta diferencia es importante: un catálogo puede pasar SHACL y seguir necesitando revisión humana de redacción, licencias o pertinencia. Aun así, SHACL aporta una base objetiva y reproducible para

defender que el modelo cumple un contrato externo.

El estado actual de las prácticas de interoperabilidad favorece este tipo de validación automática. Frente a una revisión manual de documentos, una *shape* versionada permite repetir la comprobación en local, en Continuous Integration / Continuous Delivery (CI/CD), en una consola web o en un entorno cloud. Por eso el TFM vendoriza las *shapes* oficiales y evita descargarlas en tiempo de ejecución.

A modo de ejemplo, el listado 3.3 muestra cómo se declara, en SHACL, que todo recurso de tipo `dcatalog:Dataset` debe tener al menos un título y una descripción. Si el grafo JSON-LD exportado infringe estas restricciones, el validador devuelve una violación con severidad `sh:Violation` que indica la `sh:focusNode`, la `sh:resultPath` y el mensaje correspondiente.

```
1 @prefix sh:    <http://www.w3.org/ns/shacl#> .
2 @prefix dct:  <http://purl.org/dc/terms/> .
3 @prefix dcat: <http://www.w3.org/ns/dcat#> .
4
5 [] a sh:NodeShape ;
6   sh:targetClass dcat:Dataset ;
7   sh:property [
8     sh:path      dct:title ;
9     sh:minCount  1 ;
10    sh:severity   sh:Violation ;
11    sh:message    "Todo dcat:Dataset debe declarar al menos un dct:title."@es ;
12  ] ;
13  sh:property [
14    sh:path      dct:description ;
15    sh:minCount  1 ;
16    sh:severity   sh:Violation ;
17    sh:message    "Todo dcat:Dataset debe declarar al menos una dct:description.
18    ↪ "@es ;
19  ] .
```

Código 3.3: Fragmento ilustrativo de SHACL, serializado en Turtle, que exige título y descripción a cada Dataset DCAT.

3.7. OpenMetadata

OM es una plataforma abierta de descubrimiento, observabilidad y gobierno de metadatos. Su documentación la presenta como una solución unificada con repositorio central de metadatos, conectores de ingesta, búsqueda, linaje, calidad, colaboración y gobierno [14]. Para este TFM, sus funciones más relevantes son el catálogo técnico, la Application Programming Interface (API) Representational State Transfer (REST), las entidades de base de datos y la posibilidad de añadir etiquetas y *custom properties*.

OM funciona como punto de captura y gobierno técnico. Una ingesta de PostgreSQL registra servicios, bases de datos, esquemas, tablas y columnas. Sobre esas entidades se pueden mantener descripciones, nombres visibles, dominios, etiquetas y propiedades extendidas. La plataforma resultante no inventa activos manualmente: parte de lo descubierto por OM.

3.7.1. Gobierno centralizado frente a edición dataset por dataset

Sobre OM existen dos formas razonables de hacer que un catálogo cumpla DCAT-AP-ES. La primera es la que se obtiene de fábrica: declarar el conjunto mínimo de propiedades DCAT-AP-ES como *custom properties* y editar cada dataset desde la interfaz de OM, dataset por dataset, hasta que el grafo exportado contenga todos los campos requeridos. Este flujo es válido para una decena de tablas, pero presenta tres limitaciones cuando aumenta el volumen: el trabajo manual escala linealmente con el número de datasets, las reglas globales (publicador, licencia, taxonomía, legislación HVD) deben repetirse en cada activo, y la trazabilidad de quién editó qué se diluye en eventos de UI.

La segunda forma, que es la propuesta operativa de este TFM, consiste en *externalizar la curación funcional* a una hoja de gobierno versionada y reconciliarla contra OM desde un flujo idempotente. Una persona responsable del catálogo edita un único archivo Comma-Separated Values (CSV) en el que cada fila representa un dataset publicable; los valores comunes del catálogo (publicador, *homepage*, licencia, idioma, contacto, taxonomía temática y legislación HVD) viven como *defaults* en un YAML Ain't Markup Language (YAML) de configuración común. Conviene precisar el alcance de esos *defaults*: operan en dos niveles. Por un lado describen las propiedades del `dcatalog:Catalog`; por otro, actúan como valor por defecto heredable por cada `dcatalog:Dataset` cuando este no declara uno propio. En el caso de uso de validación, publicador, licencia, idioma y contacto son homogéneos —un único organismo, la Universidad de Castilla-La Mancha (UCLM)—, de modo que centralizarlos evita repetir y desincronizar el mismo valor en cada fila. En un despliegue real con varios organis-

mos, o con licencias, contactos y responsables distintos por dataset, esos campos pueden sobrescribirse por fila en la hoja sin tocar el núcleo: la jerarquía es *default* global del catálogo, sobrescribible a nivel de dataset. Un servicio Python fusiona ambas fuentes, sincroniza los cambios contra la API REST de OM, exporta el catálogo DCAT-AP-ES en JSON-LD y lo valida con SHACL. La consecuencia práctica es que el *mismo* estado de gobierno se aplica en una sola operación, los valores comunes no se duplican y la exportación es *determinista*: dos ejecuciones consecutivas sin cambios funcionales producen un catálogo idéntico, byte a byte, propiedad que se deriva directamente de la idempotencia del workflow (sección 3.12).

La propuesta no excluye usar las *custom properties* de OM: las requiere y las gobierna. La diferencia está en quién las gestiona. En el flujo dataset por dataset las gestiona un operador humano; en el flujo centralizado las gestiona el servicio Python a partir de una fuente declarativa, y OM se reserva como sistema de captura técnica, búsqueda y autoridad de identificadores.

3.8. Mercado de herramientas de gobierno alineadas con DCAT-AP-ES

La adopción reciente de DCAT-AP-ES ha llevado a los principales fabricantes de plataformas de gobierno de datos a anunciar soporte específico para este perfil. La oferta puede agruparse en tres bloques con propiedades distintas: plataformas comerciales de gobierno con módulo DCAT-AP-ES, soluciones de portal de datos abiertos y plataformas de metadatos open source.

Plataformas comerciales de gobierno con módulo DCAT-AP-ES. Productos como Anjana Data publican configuraciones alineadas con DCAT-AP-ES [15], ofreciendo flujos editoriales, aprobaciones, jerarquías de dominios y publicación gestionada hacia portales nacionales. El valor que aportan es de producto completo: tienen UI pulida, gestión de roles, multi-tenant, ciclo de vida de activos, soporte y certificaciones. Su contrapartida es el coste de licencia, la complejidad de despliegue y, en términos académicos, la opacidad: las reglas de mapeo y validación viven dentro del producto y no son reproducibles fuera de él.

Portales de datos abiertos. Sistemas como CKAN o las extensiones de portal nacional (datos.gob.es, data.europa.eu) describen el dataset en términos de DCAT-AP y DCAT-AP-ES desde el lado del consumidor: federan, agregan y publican. No están diseñados para gobernar

metadatos técnicos desde la fuente; asumen que los catálogos productores ya entregan descripciones conformes. Por eso este TFM descarta CKAN como flujo operativo principal y lo mantiene únicamente como destino federable.

Plataformas open source de metadatos. Soluciones como OpenMetadata [14], DataHub [16] o Amundsen [17] ofrecen catálogo técnico, ingesta y gestión de *custom properties*, pero no incluyen un exportador DCAT-AP-ES oficial certificado ni una validación SHACL contra las *shapes* HVD. Para cumplir el perfil hay que extenderlas con código propio, que es precisamente la aportación de este TFM sobre OM.

3.8.1. Posicionamiento de la Plataforma de Gobierno del Dato del TFM

La Plataforma de Gobierno del Dato propuesta no pretende competir con un producto comercial. Su valor académico y técnico se sitúa en una franja concreta:

- **Reproducibilidad total:** todo el flujo (ingesta, gobierno, exportación, validación) es código versionado, sin componentes propietarios.
- **Conformidad verificable:** la validación SHACL usa las *shapes* oficiales vendorizadas, incluyendo el perfil HVD, y produce informes objetivos.
- **Gobierno centralizado y declarativo:** una hoja de gobierno CSV más defaults YAML sustituyen la edición manual dataset por dataset descrita en la sección 3.7.1.
- **Operación cerrada:** una consola web ejecuta operaciones versionadas sin duplicar reglas, ideal para perfiles no desarrolladores.

Frente a un producto comercial, el TFM acepta no cubrir flujos avanzados de aprobación, búsqueda editorial sofisticada, multi-tenant ni publicación automática hacia portales externos. A cambio, ofrece una capa abierta, auditable y formalmente validada sobre OM: el mismo perfil DCAT-AP-ES que las herramientas comerciales prometen, construido con piezas estándar (Python, API REST, JSON-LD, SHACL) y reproducible en cualquier entorno con Kubernetes local.

3.9. PostgreSQL como fuente técnica reproducible

PostgreSQL es una base de datos relacional ampliamente utilizada y adecuada para construir un caso de uso reproducible. En el mundo real, los catálogos técnicos suelen ingerir

metadatos desde múltiples motores: PostgreSQL, MySQL, sistemas cloud, data lakes o almacenes analíticos. Para el TFM, una única tecnología de origen es suficiente siempre que permita demostrar estructura técnica real: servicio, base de datos, esquemas, tablas y columnas.

La doble ingesta usada en el caso de uso de validación no pretende simular dos organizaciones distintas, sino evidenciar que la lógica de gobierno opera por identificadores completos de OpenMetadata. Dos servicios pueden contener tablas con el mismo nombre. Si el sistema mezcla filas por `schema.table`, el resultado sería incorrecto. Por eso el FQN del activo técnico se convierte en un identificador esencial.

3.10. Docker, Kind, Kubernetes y Helm

Docker proporciona contenedores: paquetes ejecutables con aplicación, dependencias y configuración de runtime [18]. Kind permite ejecutar clústeres Kubernetes locales usando nodos como contenedores Docker, lo que resulta adecuado para desarrollo, pruebas y CI [19]. Kubernetes es una plataforma portable y extensible para gestionar cargas de trabajo y servicios contenedorizados mediante configuración declarativa y automatización [20]. Helm actúa como gestor de paquetes para Kubernetes, empaquetando aplicaciones como charts y desplegándolas como releases [21].

En el estado actual de la ingeniería cloud, esta combinación es habitual para construir entornos reproducibles. No hace falta contratar un clúster gestionado para demostrar una arquitectura portable: Kind permite trabajar localmente, Kubernetes mantiene el modelo operativo y Helm aproxima el despliegue a prácticas profesionales. La limitación es evidente: no aporta alta disponibilidad ni hardening productivo. Para un TFM, esa limitación es aceptable porque el objetivo es reproducibilidad y claridad, no operación empresarial avanzada.

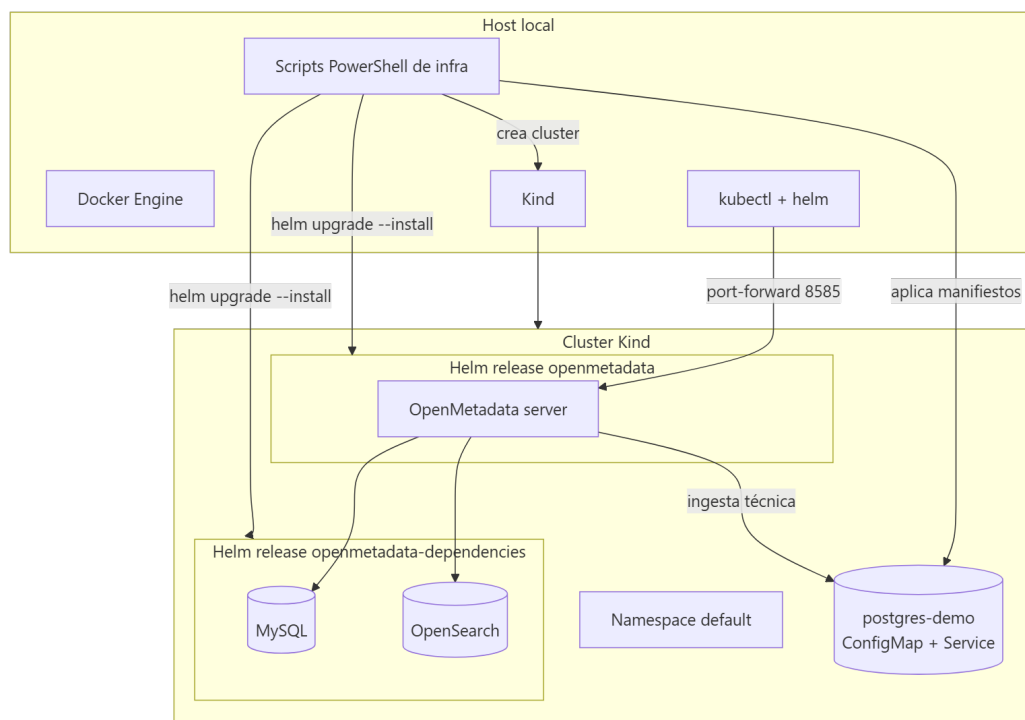


Figura 3.1: Despliegue Kubernetes reproducible usado como base del caso de uso de validación. Elaboración propia.

3.11. Consolas web operativas y experiencia de usuario

Una consola web operativa convierte comandos reproducibles en una experiencia de uso guiada. En proyectos de gobierno de datos, esto es relevante porque no todos los perfiles que revisan metadatos son desarrolladores. Una persona responsable de catálogo puede necesitar revisar la hoja de gobierno, lanzar una validación o consultar artefactos sin recordar comandos de PowerShell o Python.

La tecnología web actual facilita construir este tipo de consola con componentes reutilizables y separación entre cliente y servidor. React permite definir interfaces mediante componentes y estado [22]. Next.js añade estructura de aplicación, rutas, componentes de servidor y una capa backend integrada [23]. TypeScript aporta comprobación estática de tipos sobre JavaScript, ayudando a detectar errores antes de ejecutar el código [24].

En este TFM, la web forma parte del estado del arte aplicado porque representa una forma habitual de operar flujos técnicos desde una interfaz controlada. Sin embargo, la regla arquitectónica es estricta: la UI no contiene lógica de gobierno. Solo invoca operaciones cerradas, edita la hoja funcional y muestra resultados, logs y artefactos.

3.12. Reproducibilidad, idempotencia y *Infrastructure as Code*

La reproducibilidad es un requisito transversal del estado actual de la ingeniería de plataformas: un entorno debe poder reconstruirse de forma determinista a partir de su descripción declarativa, sin depender del estado mutable de la máquina que lo levantó por primera vez [20]. La combinación de Docker, Kind, Helm y manifiestos versionados implementa este principio: el clúster se describe como datos (YAML), no como una secuencia de pasos manuales.

La idempotencia es el complemento operativo de la reproducibilidad. Una operación es idempotente cuando aplicarla múltiples veces produce el mismo resultado que aplicarla una sola vez. En gobierno de metadatos esto es crítico: si un *workflow* de sincronización no es idempotente, repetir la ejecución duplica etiquetas, sobrescribe campos curados o genera ruido en el catálogo. OM expone una API REST [14] en la que los PATCH JSON-Patch son naturalmente idempotentes para la mayoría de operaciones; la plataforma del TFM aprovecha esta semántica para garantizar que la segunda ejecución del *workflow* no aplique cambios, propiedad que se verifica empíricamente en el capítulo 6.

Las herramientas de prueba completan este marco: pytest [25] permite expresar invariantes funcionales sobre el núcleo Python con *fixtures* reutilizables, mientras que Vitest [26] cumple el rol equivalente en la consola web Next.js. La combinación de pruebas, idempotencia y reproducibilidad de infraestructura constituye el conjunto mínimo aceptable para defender la calidad técnica de una plataforma de gobierno hoy.

3.13. Validación frente a verificación

La memoria distingue de manera consistente entre *verificación* y *validación*. *Verificación* comprueba que el sistema se ha construido correctamente según su especificación interna: pasan los tests unitarios, los tipos son consistentes, las operaciones son idempotentes. *Validación* comprueba que el sistema construido cumple un contrato externo: el catálogo exportado satisface las *shapes* SHACL oficiales de DCAT-AP-ES [12], las propiedades obligatorias del perfil HVD están presentes y los vocabularios temáticos provienen de la taxonomía Norma Técnica de Interoperabilidad de Reutilización de Recursos de Información del Sector Público (NTI-RISP) [27].

Esta distinción es relevante para una memoria académica porque permite separar lo que demuestra el TFM (validación externa contra un perfil estándar) de lo que asegura el código (verificación interna). Las dos capas se necesitan: la verificación sin validación produ-

ce software estable pero opcionalmente irrelevante; la validación sin verificación produce conformidad puntual que puede romperse en la siguiente ejecución.

3.14. Síntesis del estado del arte

El estado actual combina varias capas: gobierno del dato para definir responsabilidades, catálogos de metadatos para inventariar activos, perfiles DCAT para interoperar, SHACL para validar, OM para capturar y gobernar metadatos técnicos, Kubernetes y Helm para reproducir infraestructura, y una consola web para operar el flujo. La aportación del TFM consiste en ensamblar esas piezas con un alcance reducido, reproducible y defendible: doble ingesta PostgreSQL, gobierno funcional de tablas `gold`, exportación DCAT-AP-ES y validación SHACL, con una capa centralizada de curación que diferencia la propuesta frente a la edición *custom property* dataset por dataset descrita en la sección 3.7.1.

4. Requisitos y arquitectura de la solución

4.1. Requisitos funcionales

Los requisitos funcionales identifican capacidades observables que la plataforma debe ofrecer al usuario operador y al evaluador externo. La tabla 4.1 los enumera con identificador, descripción y criterio de aceptación verificable.

ID	Requisito	Criterio de aceptación
RF-01	Despliegue reproducible de OpenMetadata y sus dependencias.	El script <code>launch_infra.ps1</code> levanta el clúster con <code>kubect1 get pods</code> sin errores.
RF-02	PostgreSQL de referencia con capas <code>bronze/silver/gold</code> .	<code>opendata_demo_init.sql</code> aplicado; los tres esquemas son visibles.
RF-03	Doble ingesta de PostgreSQL en OpenMetadata.	Dos <code>DatabaseService</code> diferenciados con tablas <code>gold</code> .
RF-04	Selección de datasets publicables.	Solo filas con <code>publicar=si</code> aparecen en el catálogo exportado.
RF-05	Curación funcional por Fully Qualified Name (identificador completo de activo en OpenMetadata) (FQN) de tabla.	La hoja CSV identifica filas únicas por <code>table_fqn</code> .
RF-06	Sincronización idempotente con OpenMetadata.	Una segunda ejecución consecutiva aplica 0 cambios.
RF-07	Exportación a JSON-LD conforme al perfil activo.	Se genera <code>dcat_catalog.jsonld</code> con clases DCAT activas.
RF-08	Validación SHACL contra el bundle vendorizado.	Se genera <code>dcat_validation_report.ttl</code> sin violaciones bloqueantes.
RF-09	Consola web operativa para todo el flujo.	La UI ejecuta cada etapa y muestra logs y artefactos.

Tabla 4.1: Requisitos funcionales con criterio de aceptación. Elaboración propia.

4.2. Requisitos no funcionales

Los requisitos no funcionales recogen las propiedades de calidad transversales que la plataforma debe sostener (tabla 4.2). Se priorizan reproducibilidad e idempotencia por encima de capacidades habituales en sistemas productivos (alta disponibilidad, Role-Based Access Control (RBAC) avanzado, escalado), que se declaran fuera de alcance en la sección 4.3.

Atributo	Compromiso
Reproducibilidad	Cualquier ejecución parte de comandos versionados; el repositorio limpio reconstruye el entorno completo.
Idempotencia	La sincronización contra OM no duplica etiquetas, propiedades ni recursos.
Simplicidad operativa	Una única capa Python concentra reglas de negocio; Command Line Interface (CLI), scripts y web son envoltorios.
Trazabilidad	Cada artefacto se asocia a configuración, código y comando de origen.
Separación de responsabilidades	La consola web no implementa reglas; las delega en el núcleo.
Portabilidad	El despliegue es transferible entre entornos compatibles con Kubernetes y Helm.
Auditabilidad	Cada ejecución produce evidencia regenerable y consultable.

Tabla 4.2: Requisitos no funcionales y compromisos asociados. Elaboración propia.

4.3. No objetivos

Quedan deliberadamente fuera del alcance funcional la alta disponibilidad, el hardening avanzado, la integración con Single Sign-On (SSO)/Lightweight Directory Access Protocol (LDAP), el RBAC granular, los backups productivos, la observabilidad avanzada, el escalado horizontal y la cosecha activa desde CKAN. Estas exclusiones no son carencias de implementación: corresponden a decisiones de diseño que evitan que una capa adicional desplace la atención del problema principal, que es el gobierno reproducible y la conformidad DCAT-AP-ES.

4.4. Arquitectura lógica

La arquitectura se organiza en cinco capas. En lugar de describirlas con una tabla, se representan mediante un diagrama (figura 4.1), que hace explícita la responsabilidad de cada capa y la dirección de las dependencias y del flujo de datos entre ellas [28]. De abajo arriba, la fuente técnica (PostgreSQL de referencia) alimenta el catálogo técnico (OpenMetadata), del que el gobierno funcional toma los activos descubiertos; el núcleo Python fusiona hoja y defaults, aplica gobierno idempotente sobre OpenMetadata y produce el catálogo DCAT-AP-ES validado; la capa de operación (CLI, scripts y consola web) invoca ese mismo núcleo sin duplicar reglas.

Cada capa se respalda con evidencia versionada en el repositorio: la fuente técnica en

sql/opendata_demo_init.sql; el catálogo técnico en ingest_postgres_double.ps1; el gobierno funcional en gold_governance.csv y governance_defaults.yaml; el núcleo en tfm_ingestor/src/tfm_ingestor/; y la operación en scripts/infra/ y web/.

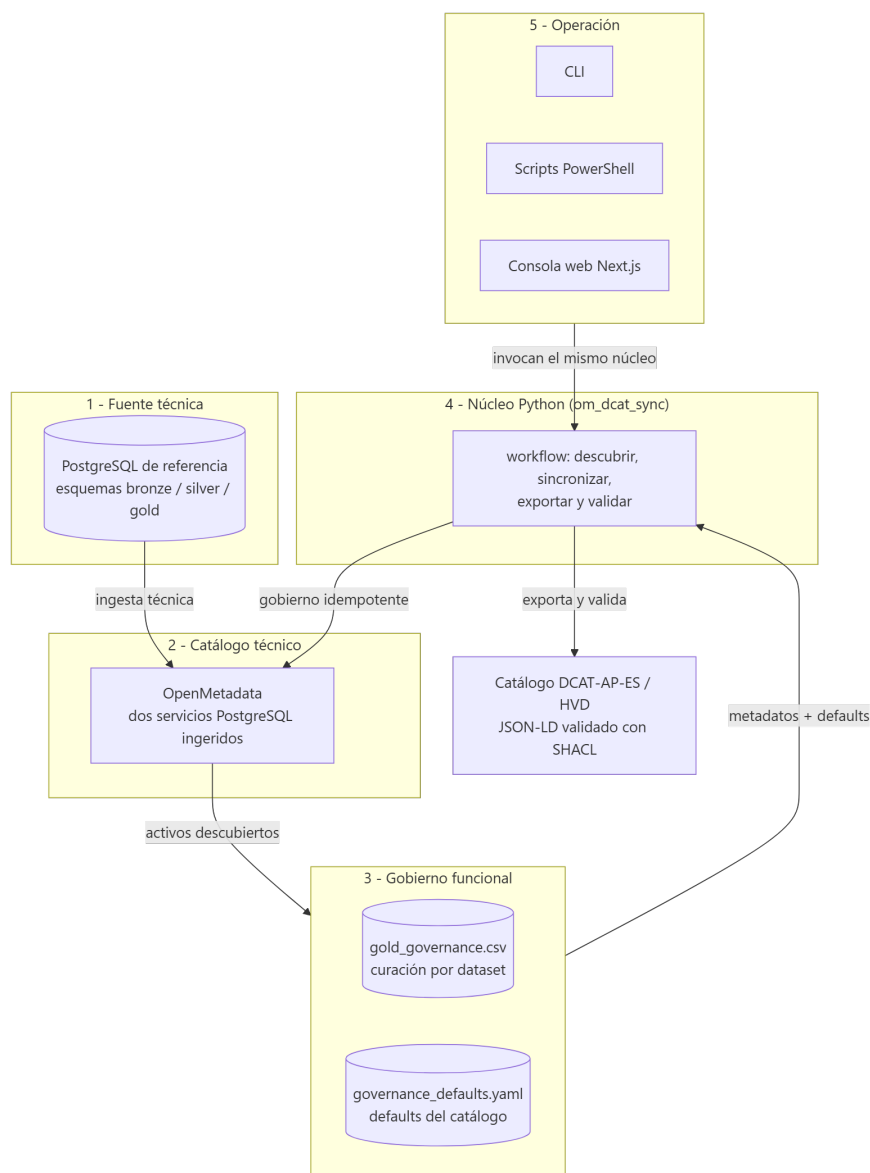


Figura 4.1: Arquitectura lógica por capas y responsabilidades: fuente técnica, catálogo técnico, gobierno funcional, núcleo Python y operación. Elaboración propia.

4.5. Arquitectura física

La vista física (figura 4.2) muestra cómo se materializan esas capas en el despliegue. Todo se ejecuta sobre un único host —portátil, VPS o nube— con Docker como motor de contenedores y un clúster Kubernetes local gestionado por Kind. OpenMetadata, sus dependencias (MySQL y OpenSearch) y el PostgreSQL de referencia se ejecutan como cargas dentro del clúster. El operario interactúa a través de la consola web y del CLI, que se comunican con OpenMetadata mediante un `port-forward` sobre la API REST autenticada con JSON Web Token (JWT). El detalle de los *values* Helm y de los manifiestos se recoge en el Anexo B.

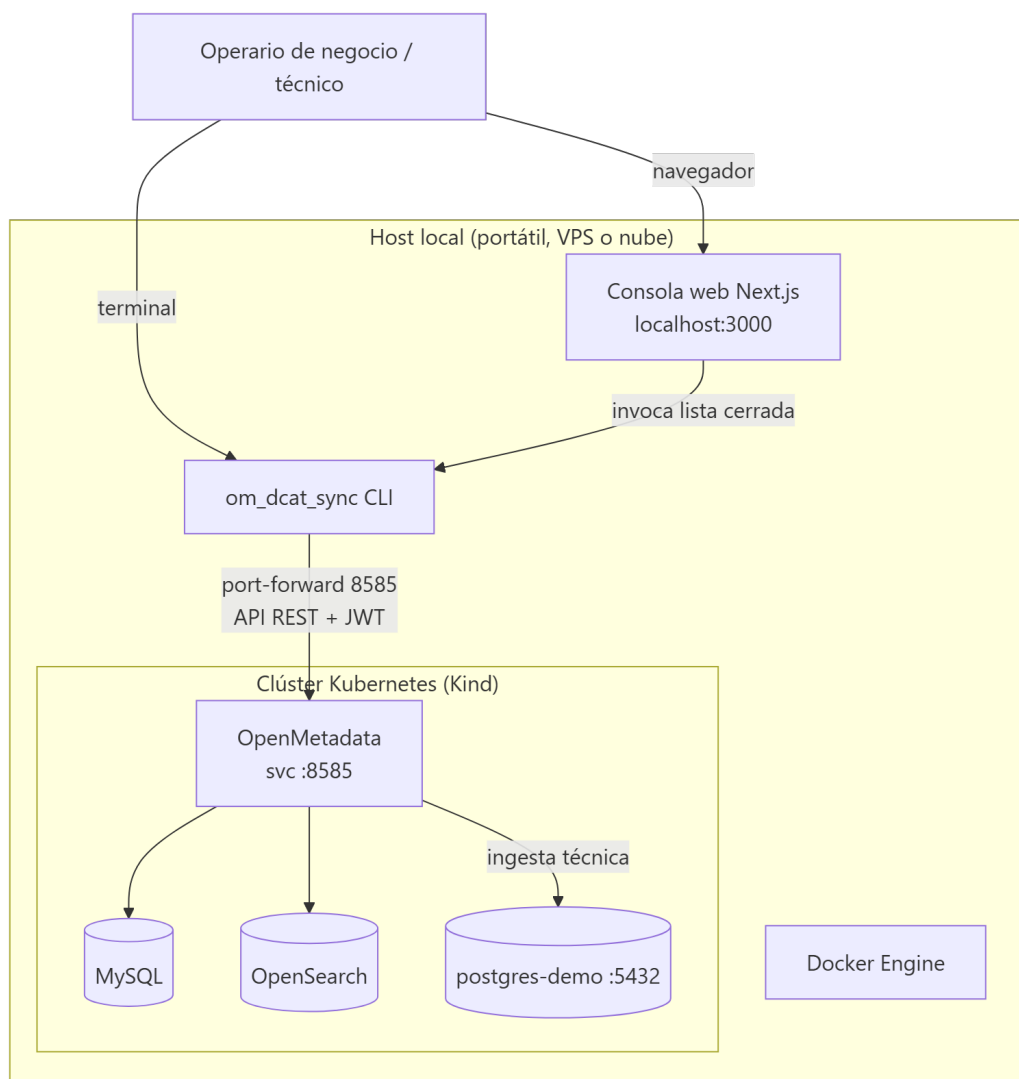


Figura 4.2: Arquitectura física de despliegue: host con Docker y clúster Kind que ejecuta OpenMetadata, sus dependencias y el PostgreSQL de referencia. Elaboración propia.

4.6. Flujo extremo a extremo

El flujo implementado sigue una secuencia cerrada. Primero se levanta la infraestructura. Después se ingiere PostgreSQL dos veces en OpenMetadata, bajo dos servicios diferenciados. A continuación se preparan etiquetas y custom properties. El workflow refresca la hoja de gobierno, permite revisar los metadatos funcionales, aplica cambios en OpenMetadata, exporta JSON-LD y ejecuta la validación SHACL.



Figura 4.3: Flujo operativo del caso de uso de validación. Elaboración propia.

La figura 4.4 muestra la transformación conceptual desde activos técnicos hasta salida validada.

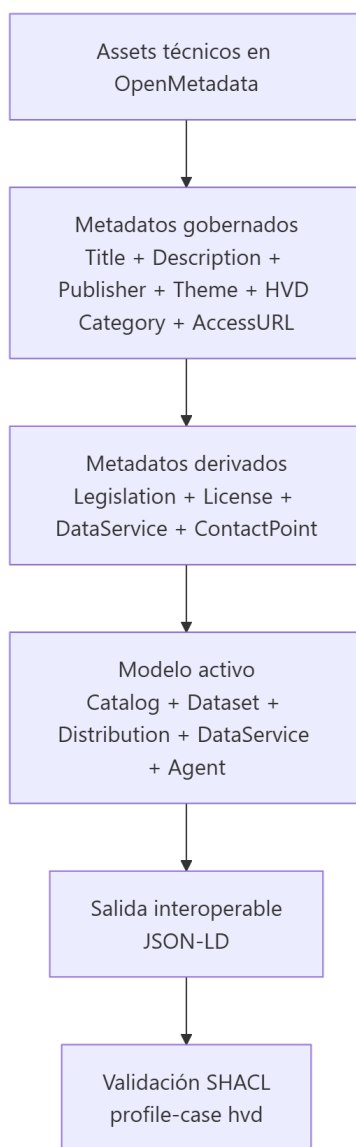


Figura 4.4: Pipeline de metadatos desde OpenMetadata hasta validación SHACL. Elaboración propia.

4.7. Decisiones de diseño

4.7.1. PostgreSQL como fuente canónica

El origen técnico se fija en PostgreSQL porque permite controlar la estructura, reproducir el entorno y demostrar descubrimiento real de activos técnicos. El SQL de referencia crea seis tablas distribuidas en tres capas. Las tablas `gold.movilidad_resumen_municipio` y `gold.agenda_cultural_publica` representan el catálogo publicable.

4.7.2. Gobierno restringido a la capa gold

La arquitectura *medallion* (bronze/silver/gold) es una convención consolidada en plataformas de datos modernas, popularizada por Databricks como patrón de zonas progresivas de refinamiento [29], que separa tres estados de madurez: datos crudos ingeridos sin transformar (bronze), datos limpios y conformados (silver) y datos refinados, agregados y listos para consumo final (gold). La plataforma gobierna *únicamente* la capa gold; las capas bronze y silver se ingieren en OM pero no se exportan como datasets DCAT-AP-ES. Esta restricción es deliberada y se apoya en cuatro razones convergentes.

Conformidad semántica con el perfil DCAT-AP-ES. El perfil exige que un `dcat:Dataset` represente un conjunto de datos «publicable», con título, descripción, publicador, temática y, en el caso HVD, categoría y legislación aplicables. Las tablas bronze y silver no satisfacen ese contrato: contienen estructuras transitorias, esquemas en evolución o columnas técnicas de ingesta que no tienen significado funcional fuera del pipeline interno. Publicarlas como datasets falsearía el catálogo y haría fracasar la validación SHACL en propiedades obligatorias.

Coherencia con el marco DAMA-DMBOK. DMBOK establece que el gobierno aplica sobre activos cuyo significado, propiedad y calidad están declarados y son estables [5]. Las capas bronze y silver son explícitamente transitorias en la arquitectura medallion y no cumplen ese requisito; gobernarlas obligaría a establecer responsables, descripciones y temáticas para datos cuya forma no es estable, generando deuda editorial sin contrapartida.

Separación entre catálogo técnico y catálogo de publicación. OM sigue actuando como inventario técnico completo: las tres capas son visibles, navegables y trazables desde su UI,

lo que mantiene la utilidad de OM como herramienta interna. La frontera se aplica únicamente en la exportación a DCAT-AP-ES: el catálogo de publicación es un subconjunto curado del catálogo técnico, no su réplica. Esta separación reproduce el comportamiento real de los portales de datos abiertos como datos.gob.es o data.europa.eu, que nunca publican el inventario técnico completo de una organización.

Foco en el valor evaluado por el tribunal. El alcance funcional declarado en la sección 4.1 cubre el ciclo completo de gobierno hasta validación SHACL. Añadir *bronze* y *silver* al gobierno multiplicaría el trabajo editorial sin añadir capacidad demostrable: el flujo, los mecanismos de idempotencia y las shapes SHACL son los mismos. La restricción concentra el esfuerzo en demostrar el ciclo completo sobre un subconjunto reducido y representativo, no en escalarlo sobre datos sin valor editorial.

La decisión, por tanto, no es una limitación de implementación sino una alineación con la semántica del perfil destino, con el marco de gobierno aplicado y con la práctica habitual de los portales de datos abiertos. La plataforma puede gobernar tablas adicionales sin cambios estructurales: basta con marcar `publicar=si` en la hoja `gold_governance.csv` para los datasets nuevos, lo que demuestra que la decisión es de alcance, no de capacidad técnica.

4.7.3. Doble ingesta como validación multi-fuente

La doble ingesta usa la misma base de referencia, pero crea dos `DatabaseService` en `OpenMetadata`: `postgres_demo_service` y `postgres_validation_service`. El objetivo no es duplicar datos por volumen, sino comprobar que el gobierno funciona cuando existen tablas con el mismo nombre en servicios distintos. Por eso la hoja funcional conserva `schema_name`, `table_name` y, sobre todo, `table_fqn`.

4.7.4. Hoja funcional para gobierno editorial

En este trabajo, *curación* designa la actividad editorial de seleccionar, describir, enriquecer y mantener los metadatos de un activo para que sean comprensibles y publicables; es el término consolidado en gestión del dato (*data curation* [5]), frente a «edición» —demasiado genérica— o a «limpieza» y «saneamiento», que aluden a la calidad del dato de negocio y no a sus metadatos. La *curación funcional* es esa misma actividad aplicada a los metadatos funcionales o de negocio de cada dataset (título, descripción, publicador, temática y categoría HVD), por oposición a los metadatos técnicos que OM captura de forma automática.

La curación funcional se concentra en `tfm_ingestor/config/gold_governance.csv`.

Esta hoja permite editar los metadatos que varían por dataset: decisión de publicación, título, descripción, publicador, temática DCAT, categoría HVD y URL de acceso. Los campos técnicos (esquema, tabla y FQN) no deben editarse manualmente salvo regeneración controlada del caso de uso. La tabla 4.3 resume las columnas; el detalle completo con criterios de validación se ofrece en el anexo C.

Columna	Tipo	Propósito
publicar	si/no	Inclusión del dataset en el catálogo exportado.
schema_name	texto	Esquema PostgreSQL (gold en el caso de uso).
table_name	texto	Nombre técnico de la tabla.
table_fqn	texto	Identificador completo en OM; clave única.
titulo_dataset	texto	Título funcional editorial.
descripcion_dataset	texto	Descripción editorial del dataset.
publicador	texto	Nombre del organismo publicador.
tematica_dcat	enum NTI-RISP	Temática NTI-RISP del dataset.
categoria_hvd	enum HVD	Categoría del Reglamento (UE) 2023/138.
access_url_distribucion	URL	URL pública de la distribución.

Tabla 4.3: Columnas de la hoja funcional `gold_governance.csv`. Elaboración propia.

El campo `publicar` merece una explicación de gobierno y no de mero filtrado técnico. La exportación del catálogo es idempotente y, por defecto, recorre el total de activos gobernados —datasets, distribuciones y servicios de datos—. El valor `si/no` introduce un control editorial sobre qué entra realmente en el catálogo DCAT-AP-ES publicado. Esa distinción alinea la plataforma con el principio DAMA de gestionar el ciclo de vida del dato [5]: permite diferenciar los datasets publicables de aquellos que permanecen privados, en estado de borrador o catalogados internamente pero todavía no federables. De este modo, una misma hoja puede reflejar estados de publicación distintos por dataset y acompañar su evolución hacia la federación sin perder ni trazabilidad ni reproducibilidad.

4.7.5. Configuración global por YAML

Los metadatos globales del catálogo se derivan de `governance_defaults.yaml`: nombre del catálogo, descripción, publicador, URI de organismo, página principal, idioma, licencia, legislación HVD, contacto y bases de URLs. Estos valores operan como *defaults* a dos niveles: describen el `dcat:Catalog` y, además, sirven de valor por defecto heredable por cada `dcat:Dataset`. Esta separación evita repetir información común en cada dataset y reduce errores de consistencia; cuando un dataset concreto requiere un publicador, una licencia, un idioma o un contacto propios —habitual cuando coexisten varios organismos o responsables—, la hoja funcional puede sobrescribir el default global a nivel de fila sin modificar el núcleo. El listado 4.1 muestra el perfil operativo que conecta esos defaults con la hoja CSV y fija el caso HVD como modo de validación.

```
1 defaults_path: "tfm_ingestor/config/governance_defaults.yaml"
2 rules_path:    "tfm_ingestor/config/mapping_rules.yaml"
3 sheet_path:    "tfm_ingestor/config/gold_governance.csv"
4
5 workflow:
6   profile_case: "hvd"
7   allow_warnings: false
8   refresh_sheet: true
9   export_output: "dcat_catalog.jsonld"
10  report_output: "tmp_pytest/dcat_validation_report.ttl"
```

Código 4.1: Perfil operativo `operational_profile.yaml` que enlaza defaults, reglas, hoja y caso de validación.

4.7.6. Núcleo único de lógica

La regla arquitectónica principal es que la lógica de gobierno vive en Python. El CLI, los scripts y la web invocan ese núcleo. Esta decisión evita que la interfaz web acabe definiendo reglas paralelas y facilita probar la solución con `pytest`. El listado 4.2 muestra la firma del endpoint canónico, una `dataclass` inmutable (`frozen=True`) que fija de manera explícita todos los parámetros del workflow: rutas de configuración, conexión a OM, modo *dry-run*, refresco de hoja, salidas de exportación y caso de validación. El término *dry-run* (literalmente «ejecución en seco») designa una ejecución que calcula y muestra el plan de cambios sin aplicarlos sobre OM. Se conserva en inglés porque es el nombre literal de la opción del CLI (`--dry-run`) y un estándar de facto en herramientas de línea de comandos e *Infrastructure*

as Code; traducirlo desincronizaría la memoria del comando real que ejecuta el operario.

```
1 from dataclasses import dataclass
2
3 @dataclass(frozen=True)
4 class WorkflowRunConfig:
5     defaults_path: str
6     rules_path: str
7     sheet_path: str
8     base_url: str
9     token: str | None
10    limit: int = 1000
11    dry_run: bool = False
12    refresh_sheet: bool = True
13    export_output: str = "dcat_catalog.jsonld"
14    report_output: str | None = None
15    allow_warnings: bool = False
16    profile_case: str = "hvd"
17    plan_output: str | None = None
18    tables_input_path: str | None = None
19    skip_export: bool = False
20    skip_validate: bool = False
21
22
23 def run_workflow(config: WorkflowRunConfig) -> dict[str, Any]:
24     defaults = load_defaults(config.defaults_path)
25     rules = load_rules(config.rules_path)
26     api = OpenMetadataApi(base_url=config.base_url, jwt_token=config.token)
27     tables = discover_tables(api=api, limit=config.limit)
28     # 1) refresca hoja 2) sincroniza gobierno 3) exporta DCAT 4) valida SHACL
29     ...
```

Código 4.2: Configuración inmutable del workflow canónico en workflow_service.py.

4.8. Modelo activo DCAT-AP-ES

El modelo activo del TFM cubre las clases `dcat:Catalog`, `dcat:Dataset`, `dcat:Distribution`, `dcat:DataService` y `foaf:Agent`. OpenMetadata mantiene únicamente los metadatos que conviene gobernar manualmente: `displayName`, `description`, `dcat_publisher_`

name, dcat_hvd_category, dcat_access_url y tags dcat_theme . *. El resto se deriva durante la exportación.

Esta decisión prioriza el camino completo sobre la cobertura exhaustiva. El alcance funcional declarado en el capítulo 2 no incluye los metadatos opcionales que no inciden en la conformidad HVD; modelarlos exigiría duplicar trabajo editorial sin aportar valor adicional al ciclo de validación, y queda explícitamente reservado como evolución posterior. La plataforma demuestra, por tanto, un camino completo y validable para el subconjunto obligatorio y operativo del caso HVD.

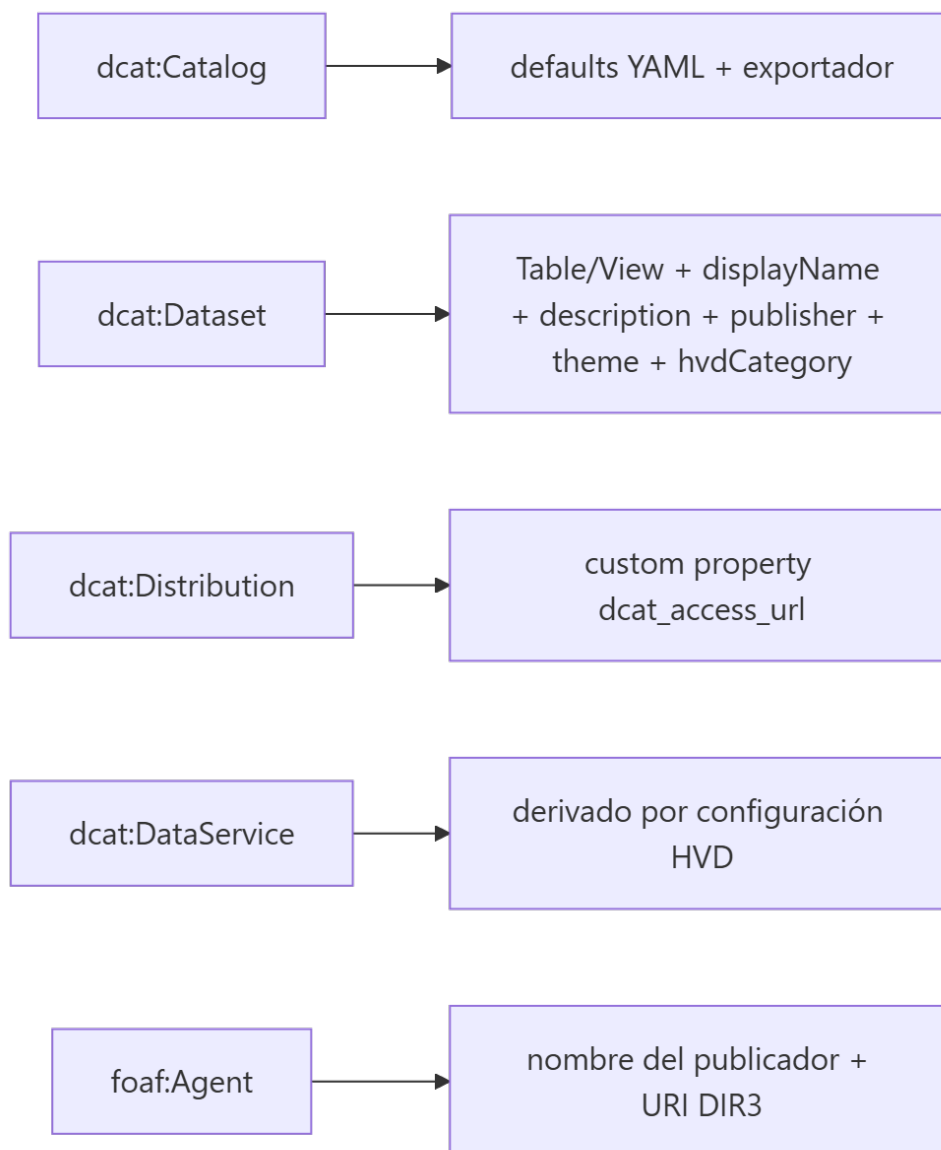


Figura 4.5: Mapeo activo entre clases DCAT-AP-ES y elementos de la plataforma. Elaboración propia.

5. Implementación

Este capítulo describe cómo se materializan en código las decisiones de diseño formuladas en el capítulo 4. La narrativa sigue el orden lógico de la plataforma: organización del repositorio, fuente técnica reproducible, despliegue de OM en Kubernetes, ingesta técnica doble, configuración funcional, núcleo Python, exportación DCAT-AP-ES, validación SHACL y consola web operativa. Los comandos exactos para reproducir cada bloque están recogidos en el Anexo A, y el recorrido visual completo desde la consola web en el Anexo F.

5.1. Organización del repositorio

El repositorio sigue una organización deliberada para que cada bloque sea identificable y reproducible. El árbol del listado 5.1 muestra los directorios canónicos en su jerarquía real, y la tabla 5.1 resume la responsabilidad funcional de cada uno.

Bloque	Responsabilidad
sql/	Activo técnico fuente del caso de uso de validación.
k8s/	Despliegue declarativo de OM y dependencias.
scripts/infra/	Operaciones reproducibles de infraestructura.
tfm_ingestor/	Lógica de negocio del gobierno y la exportación.
web/	Consola operativa sobre la lógica anterior.
docs/	Fuente narrativa y de decisiones.
tmp_pytest/	Evidencias regenerables.

Tabla 5.1: Bloques principales del repositorio. Elaboración propia.



Figura 5.1: Árbol canónico del repositorio del TFM. Elaboración propia.

5.2. Fuente técnica reproducible: PostgreSQL en tres capas

La fuente técnica es un PostgreSQL desplegado en el mismo clúster Kubernetes que OM, inicializado a partir de `sql/opendata_demo_init.sql` [30]. El SQL crea tres esquemas alineados con el patrón *medallion* (bronze/silver/gold), de modo que el caso de uso de validación dispone de datos sintéticos verosímiles sin depender de descargas externas. Únicamente el esquema *gold* se considera publicable: las capas *bronze* y *silver* se ingieren en el catálogo técnico para mantener la cadena de transformaciones visible, pero no se exportan como datasets DCAT-AP-ES por las razones de conformidad semántica, alineamiento con DMBOK y separación entre catálogo técnico y catálogo de publicación que se argumentan en la sección 4.7.2.

El listado 5.1 muestra la creación de la tabla `gold.agenda_cultural_publica`, que después se convierte en uno de los datasets del catálogo DCAT-AP-ES. La estructura es suficientemente expresiva (claves temporales, dimensiones territoriales, métricas) para que el catálogo exportado contenga campos significativos al validar con SHACL.

```

1 CREATE SCHEMA IF NOT EXISTS gold;
2
3 CREATE TABLE IF NOT EXISTS gold.agenda_cultural_publica (
4     evento_id      BIGINT      PRIMARY KEY,
5     fecha          DATE        NOT NULL,
6     municipio      TEXT        NOT NULL,
7     categoria      TEXT        NOT NULL,

```

```

8      titulo          TEXT          NOT NULL,
9      asistencia_est  INTEGER       CHECK (asistencia_est >= 0),
10     actualizado_en  TIMESTAMPTZ   DEFAULT now()
11 );

```

Código 5.1: Fragmento de `sql/opendata_demo_init.sql`: tabla publicable de la capa gold.

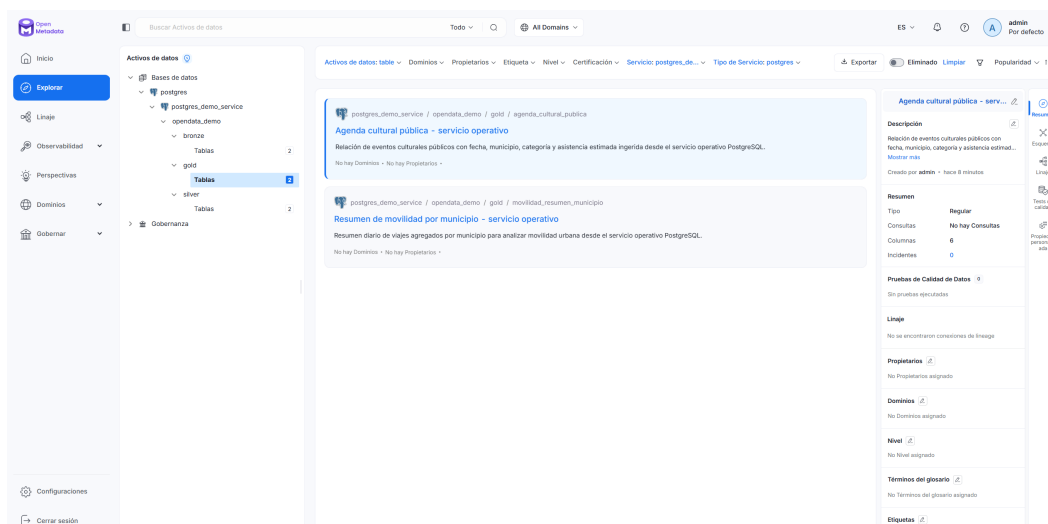


Figura 5.2: Esquemas bronze, silver y gold del PostgreSQL de referencia, vistos desde la UI de OM. Elaboración propia.

5.3. Despliegue reproducible en Kubernetes

El despliegue local utiliza Kind como motor de clúster Kubernetes encima de Docker [18, 19, 20]. OM y sus dependencias (MySQL y OpenSearch) se instalan mediante Helm [21] usando los *values* canónicos del repositorio. La figura 5.3 muestra los contenedores Docker reales del entorno y la figura 5.4 los *pods* resultantes en el clúster.

```

1 openmetadata:
2   config:
3     elasticsearch:
4       host: opensearch
5       port: 9200

```

```

6 database:
7   host: mysql
8   port: 3306
9   driverClass: com.mysql.cj.jdbc.Driver

```

Código 5.2: Fragmento de `k8s/openmetadata.values.yaml`: configuración mínima del release Helm.

El script `scripts/infra/launch_infra.ps1` encadena la creación del clúster, la instalación de los charts y los port-forward necesarios para exponer la UI de OM en `localhost:8585` y el PostgreSQL de referencia en `localhost:5432`. La idempotencia se garantiza mediante `helm upgrade --install` y comprobaciones con `kubectl get` previas a cada paso destructivo. Los *values* Helm canónicos y notas operativas adicionales se describen en el Anexo B.

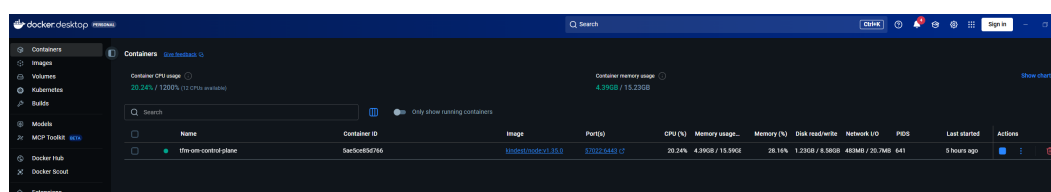


Figura 5.3: Contenedores Docker que materializan el clúster Kind y los nodos auxiliares. Elaboración propia.

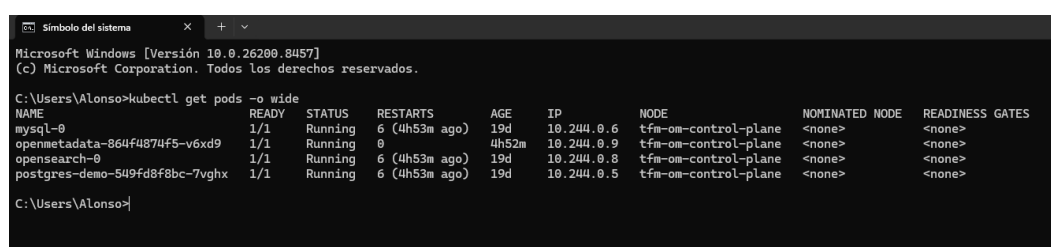


Figura 5.4: Pods desplegados en el clúster Kind: OpenMetadata, MySQL, OpenSearch y PostgreSQL de referencia. Elaboración propia.

5.4. Doble ingesta técnica

La ingesta se ejecuta dos veces sobre la misma base PostgreSQL, registrando dos DatabaseService diferenciados en OM: `postgres_demo_service` y `postgres_validation_service`. Este patrón demuestra que la lógica de gobierno opera sobre FQNs, no sobre nombres cortos de tabla; dos tablas con idéntico nombre funcional procedentes de servicios diferentes generan datasets independientes en el catálogo final.

La ingesta se invoca desde `scripts/infra/ingest_postgres_double.ps1`, que llama a los *workflows* de ingesta de OM [14] con dos configuraciones diferenciadas. El resultado es visible en la UI de OM (figura 5.5): cuatro tablas publicables (dos por servicio), cada una con su FQN único.

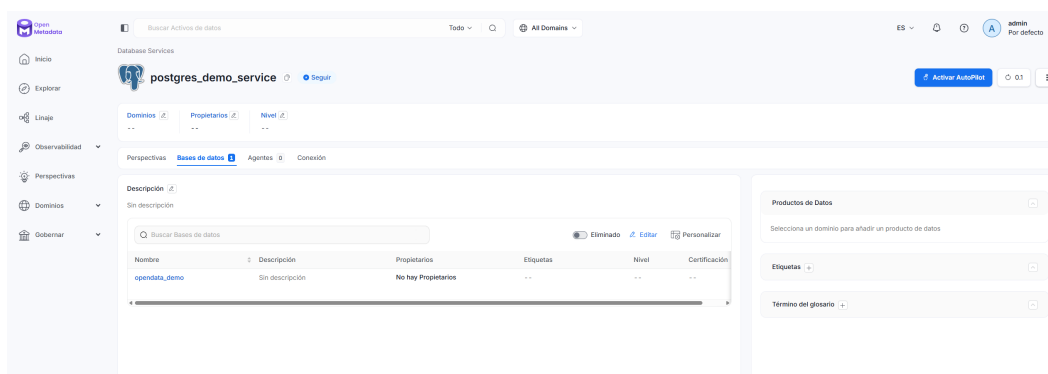


Figura 5.5: Servicios de base de datos ingeridos en OpenMetadata tras la doble ingesta. Elaboración propia.

5.5. Núcleo Python: paquete `om_dcat_sync`

El paquete `tfm_ingestor` (alias importable: `om_dcat_sync`) concentra toda la lógica de gobierno, exportación y validación. Está estructurado en módulos con responsabilidades acotadas, conforme al principio de núcleo único declarado en el capítulo 4. La figura 5.6 resume su composición y los principales módulos están listados en la tabla 5.2.

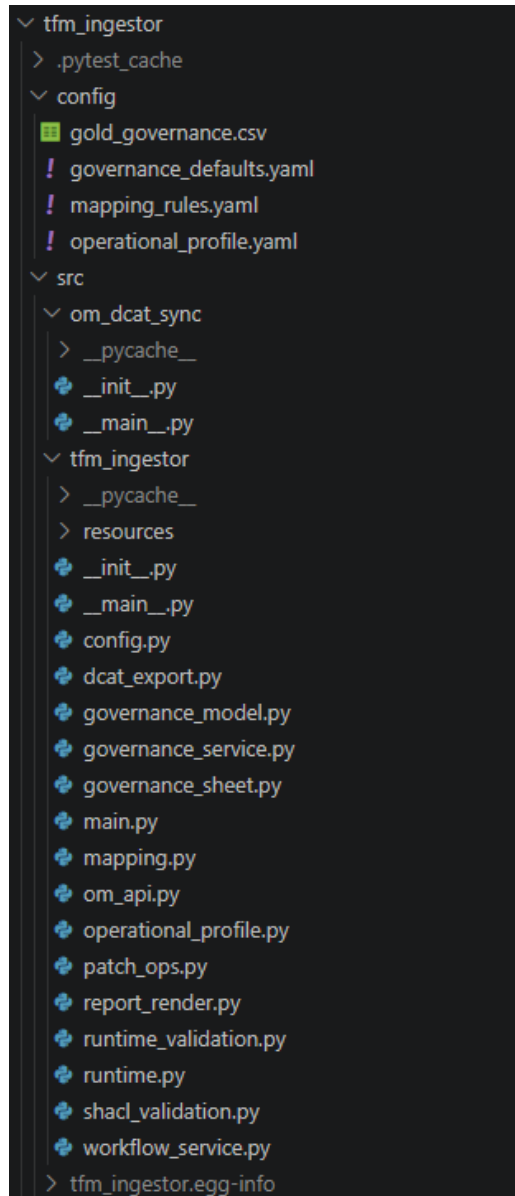


Figura 5.6: Estructura interna del paquete `om_dcat_sync`. Elaboración propia.

Módulo	Responsabilidad
workflow_service.py	Orquestador canónico del workflow end-to-end.
governance_service.py	Sincroniza gobierno entre hoja CSV y OM.
governance_sheet.py	Lectura, escritura y validación de la hoja CSV.
om_api.py	Ciente REST ligero para la API de OM.
dcat_export.py	Construye el grafo DCAT-AP-ES y lo serializa en JSON-LD.
shacl_validation.py	Aplica las <i>shapes</i> vendorizadas con pySHACL.
config.py	Carga defaults YAML y reglas de mapeo.
operational_profile.py	Une defaults, reglas y caso de validación.
runtime_validation.py	Verifica coherencia del estado vivo de OM.
main.py	CLI basado en argparse.

Tabla 5.2: Módulos principales de tfm_ingestor. Elaboración propia.

5.5.1. Configuración inmutable y endpoint

El módulo `workflow_service.py` define una `dataclass` inmutable (`frozen=True`) [31] que fija todos los parámetros del workflow antes de su ejecución, evitando estados mutables compartidos entre llamadas. El listado 5.3 amplía el listado 4.2 con la secuencia operativa real.

```

1 def run_workflow(config: WorkflowRunConfig) -> dict[str, Any]:
2     defaults = load_defaults(config.defaults_path)
3     rules    = load_rules(config.rules_path)
4     api      = OpenMetadataApi(base_url=config.base_url, jwt_token=config.token)
5     tables   = discover_tables(api=api, limit=config.limit)
6
7     # 1) Refrescar la hoja desde activos descubiertos sin perder curación previa
8     if config.refresh_sheet:
9         existing = load_governance_sheet(config.sheet_path) if Path(config.
↪ sheet_path).exists() else []
10        generate_governance_sheet(tables, defaults, config.sheet_path,
↪ existing_rows=existing,
11                                tags_by_prefix=rules.table_tags_by_prefix)
12
13    # 2) Sincronizar gobierno hacia OpenMetadata (idempotente)
14    plan = run_governance_sync(api=api, sheet_path=config.sheet_path,
15                              defaults=defaults, rules=rules, dry_run=config.
↪ dry_run)
16
17    # 3) Exportar el catálogo DCAT-AP-ES y 4) validar con SHACL
18    if not config.skip_export:
19        export_catalog(api=api, defaults=defaults, output=config.export_output)

```

```

20     if not config.skip_validate:
21         export_and_validate_catalog(profile_case=config.profile_case,
22                                   report_output=config.report_output,
23                                   allow_warnings=config.allow_warnings)
24     return {"plan": plan, "sheet_refreshed": config.refresh_sheet}

```

Código 5.3: Esqueleto de `run_workflow` en `workflow_service.py`.

5.5.2. Sincronización idempotente de gobierno

`governance_service.py` mapea cada fila de la hoja CSV a operaciones PATCH o POST sobre la entidad correspondiente en OM usando su FQN. La idempotencia se materializa por dos vías: las propiedades gestionadas (`MANAGED_DCAT_CUSTOM_PROPERTIES`) solo se reescriben si cambian, y las etiquetas `dcatalog_theme.*` se aplican como conjuntos, nunca como acumulación lineal. La consecuencia operativa es que ejecutar el workflow dos veces consecutivas produce cero cambios en la segunda ejecución, lo que constituye una de las métricas de validación documentadas en el capítulo 6.

El resultado del gobierno es directamente observable en OM: tras el workflow, cada tabla gold publicable conserva sus propiedades personalizadas DCAT (`dcatalog_publisher_name`, `dcatalog_hvd_category`, `dcatalog_access_url`) y sus etiquetas `dcatalog_theme.*`, que son la materia prima que el exportador transforma en el grafo DCAT-AP-ES (figura 5.7).

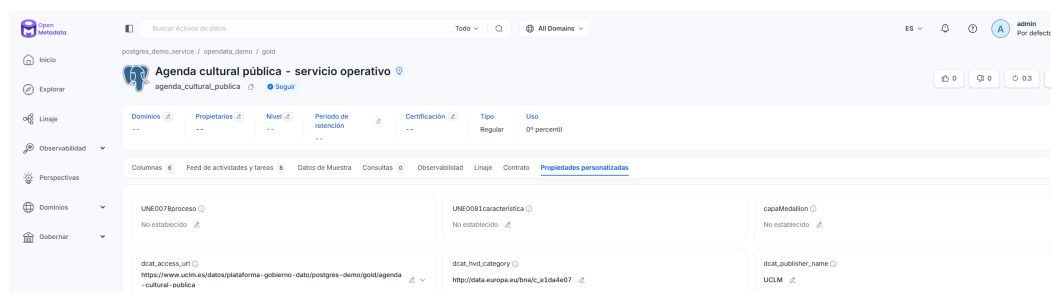


Figura 5.7: Propiedades personalizadas DCAT y etiquetas `dcatalog_theme.*` aplicadas a una tabla gold en OpenMetadata tras ejecutar el workflow. Elaboración propia.

5.5.3. Exportador DCAT-AP-ES

`dcat_export.py` compone el grafo conceptual a partir de las clases activas: `dcat:Catalog`, `dcat:Dataset`, `dcat:Distribution`, `dcat:DataService` y `foaf:Agent`. La construcción se hace en Python puro mediante diccionarios anidados que respetan el contexto JSON-LD; cuando es necesario manipular el grafo (normalización, deduplicación), se delega en `RDFLib` [32].

El listado 5.4 muestra el contexto JSON-LD canónico utilizado: prefijos estables, idioma por defecto en español y enlace a vocabularios temáticos de NTI-RISP [27].

```

1 "@context": {
2   "@language": "es",
3   "dcat": "http://www.w3.org/ns/dcat#",
4   "dct": "http://purl.org/dc/terms/",
5   "foaf": "http://xmlns.com/foaf/0.1/",
6   "skos": "http://www.w3.org/2004/02/skos/core#",
7   "vcard": "http://www.w3.org/2006/vcard/ns#",
8   "hvd": "http://data.europa.eu/r5r/applicableLegislation/",
9   "tema": "http://datos.gob.es/kos/sector-publico/sector/"
10 }
```

Código 5.4: Contexto JSON-LD canónico generado por `dcat_export.py`.

5.5.4. Validación SHACL del catálogo exportado

`shacl_validation.py` aplica el bundle vendorizado en `tfm_ingestor/resources/shacl/` con `pySHACL` [33]. El bundle está congelado en el commit `f2c8a888` de `datosgobes/DCAT-AP-ES` [12] para evitar dependencias de red y garantizar reproducibilidad entre ejecuciones. El validador devuelve un informe Turtle estructurado por `sh:focusNode`, `sh:resultPath`, `sh:severity` y `sh:resultMessage`; el módulo lo postprocesa para filtrar `sh:Warning` cuando se pasa `--allow-warnings`.

5.6. Hoja funcional de gobierno y defaults globales

La curación funcional se concentra en `tfm_ingestor/config/gold_governance.csv`, cuya estructura columnar se describe en la tabla 4.3 y se detalla con criterios de validación en el Anexo C. El uso de CSV con separador `;` no es accidental: facilita la edición en hojas de cálculo locales sin Excel y, sobre todo, hace que el diff en Git sea legible para revisión

editorial. Los defaults globales viven en `governance_defaults.yaml`, que centraliza publicador, homepage, idioma, licencia, contacto y la legislación HVD aplicable (Reglamento de Ejecución (UE) 2023/138 [34]).

La figura 5.8 muestra la pantalla *Gobierno* de la consola web, que actúa sobre la misma hoja CSV y aplica las mismas reglas de validación que el CLI. El uso real de esta pantalla durante una sesión completa se narra en la sección F.5 del Anexo F.

5.7. Consola web operativa Next.js

La consola web se construye con Next.js [23], React [22] y TypeScript [24], organizada en rutas que mapean uno-a-uno con las etapas del workflow: *Infraestructura*, *Ingesta*, *Gobierno*, *Workflow*, *DCAT*, *Estado vivo*, *Validación*, *SHACL*, *Artefactos*, *Ejecuciones*. La arquitectura interna respeta la regla de núcleo único: la UI nunca habla directamente con OM ni reimplementa reglas, sino que invoca una lista cerrada de operaciones de servidor (`operations.ts`) que a su vez ejecutan el CLI o los scripts PowerShell. La descripción operativa pantalla a pantalla, incluyendo los artefactos producidos en cada paso, se desarrolla en el Anexo F; los comandos equivalentes desde CLI y `pnpm` están recogidos en el Anexo E.

Figura 5.8: Pantalla *Gobierno* de la consola web: edición de la hoja `gold_governance.csv`. Elaboración propia.

Cada ejecución crea un *job* versionado en `state/web_jobs/` con identificador único, log completo, código de salida, duración y artefactos generados. El listado 5.5 muestra la

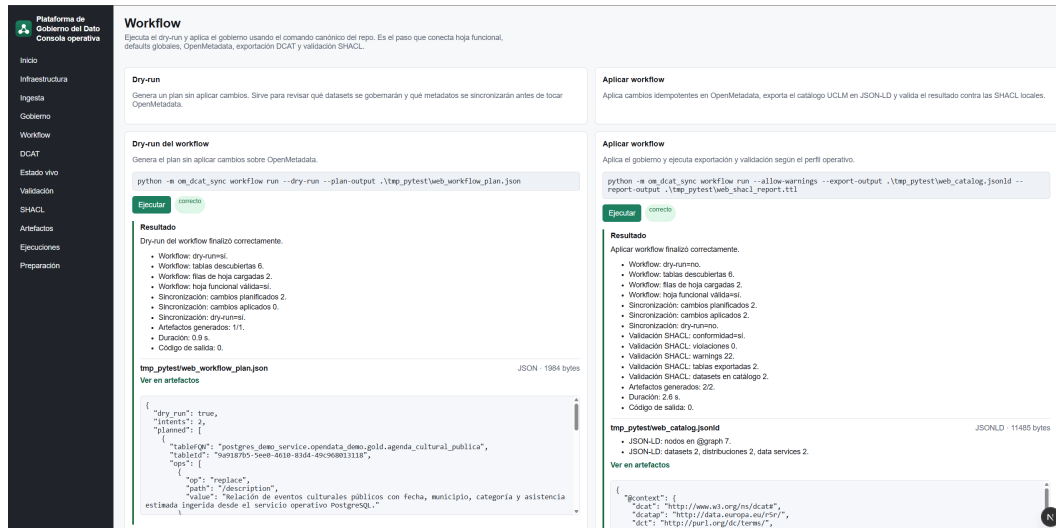


Figura 5.9: Pantalla *Workflow*: lanzamiento de dry-run y apply con resumen de cambios. Elaboración propia.

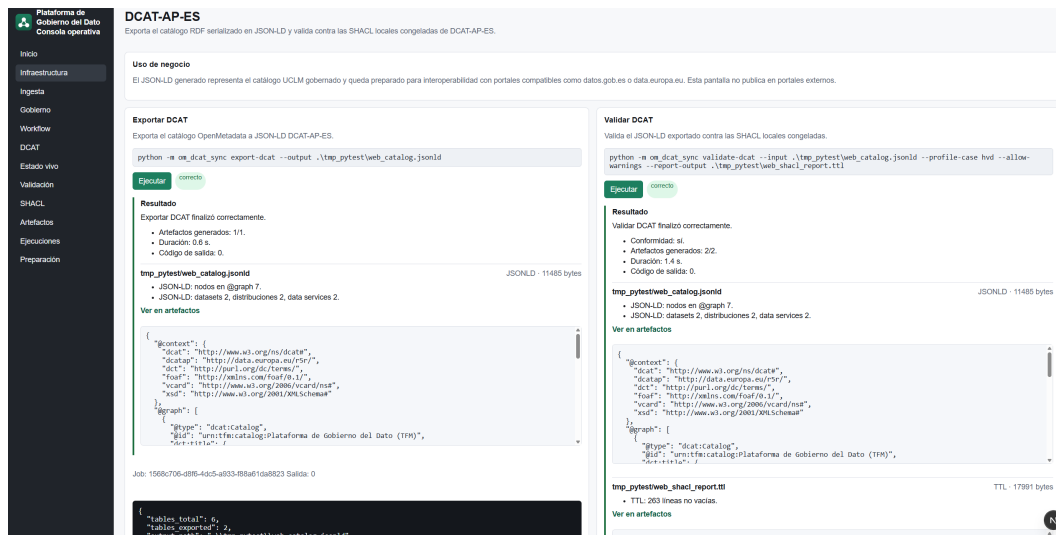


Figura 5.10: Pantalla *DCAT*: exportación y previsualización del catálogo JSON-LD. Elaboración propia.

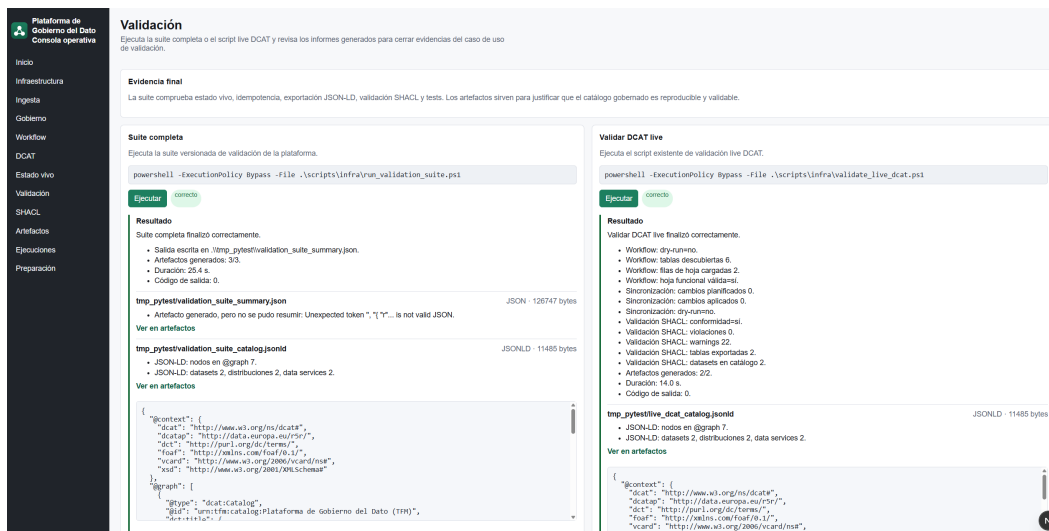


Figura 5.11: Pantalla *Validación*: ejecución de la suite reproducible y resumen de resultados. Elaboración propia.

estructura de un *job* persistido, que la pantalla *Ejecuciones* consume para renderizar el historial.

```

1 {
2   "id":          "job-2026-05-18T10-43-12-workflow-run",
3   "operation":   "workflow.run",
4   "status":      "ok",
5   "started_at":  "2026-05-18T10:43:12Z",
6   "duration_s":  47.8,
7   "exit_code":   0,
8   "summary":     { "tables": 4, "changes_applied": 0, "shacl_conformant": true },
9   "artifacts":   ["tmp_pytest/dcat_catalog.jsonld",
10                  "tmp_pytest/dcat_validation_report.ttl"]
11 }

```

Código 5.5: Estructura de un *job* persistido en `state/web_jobs/`.

5.8. Resumen de implementación

La implementación cumple las tres reglas duras declaradas en el capítulo 4: una única lógica de negocio en Python, idempotencia comprobable y separación estricta entre datos

técnicos, curación funcional y derivaciones automáticas. El capítulo siguiente describe cómo se valida formalmente este ensamblaje y qué evidencias produce.

6. Validación y resultados

Este capítulo evalúa la corrección de la salida producida por la plataforma. Complementa el recorrido operativo *end-to-end* documentado en el Anexo F, donde se narran las pantallas de la consola web y los artefactos producidos en cada paso. Aquí se mide qué garantiza esa salida; allí se muestra cómo se obtiene paso a paso.

6.1. Estrategia de validación

La validación combina tres niveles:

1. pruebas automatizadas de módulos Python y servidor web;
2. validación estructural del estado vivo de OpenMetadata;
3. validación semántica del catálogo JSON-LD mediante SHACL.

Esta estrategia evita depender de una comprobación visual de la interfaz. Cada resultado debe poder reproducirse mediante comandos versionados y artefactos generados.

6.2. Pruebas automatizadas

El repositorio contiene pruebas `pytest` para configuración, reglas de mapeo, hoja funcional, exportación DCAT, validación SHACL, validación runtime y workflow. La app web incluye tests para operaciones, lectura y escritura de CSV, resumen de jobs, artefactos y redacción de secretos. Los comandos exactos de ejecución para reproducir esta capa de pruebas figuran en el Anexo E.

Los comandos principales se muestran en el listado 6.1.

```
1 $env:PYTEST_DISABLE_PLUGIN_AUTOLOAD="1"; python -m pytest
2 cd .\web
3 pnpm test
```

Código 6.1: Ejecución de la suite pytest del núcleo Python y de los tests vitest de la consola web.

Estas pruebas cubren la lógica interna sin exigir que el tribunal inspeccione manualmente cada módulo.

6.3. Validación estructural del estado vivo

La validación runtime compara la instancia de OpenMetadata con el contrato esperado. Comprueba que existan servicio, base de datos, esquemas, tablas y columnas definidos por `sql/opendata_demo_init.sql`; también comprueba que los datasets gold publicados tengan los metadatos de gobierno esperados según la hoja funcional.

El comando canónico se muestra en el listado 6.2.

```
1 python -m om_dcat_sync validate-runtime --strict `
2     --output tmp_pytest/runtime_validation_report.json
```

Código 6.2: Validación estructural del estado vivo de OpenMetadata desde el CLI `om_dcat_sync`.

Este nivel valida trazabilidad entre la fuente técnica, OpenMetadata y la configuración funcional.

6.4. Validación SHACL

La validación SHACL usa el caso `hvd` de DCAT-AP-ES. Según la documentación oficial, este caso incluye las restricciones generales y las formas específicas para conjuntos de datos de alto valor [12]. En el repositorio, esas shapes están vendorizadas en `tfm_ingestor/src/tfm_ingestor/resources/shacl/`, junto con un manifiesto que fija el origen.

La validación puede ejecutarse de forma aislada (listado 6.3).

```
1 python -m om_dcat_sync validate-dcat --profile-case hvd --allow-warnings
```

Código 6.3: Validación SHACL aislada del catálogo JSON-LD ya exportado, caso HVD.

La ruta recomendada, sin embargo, es integrarla dentro del workflow completo (listado 6.4), que ejecuta sincronización, exportación y validación en un único paso idempotente.

```
1 python -m om_dcat_sync workflow run --allow-warnings
```

Código 6.4: Workflow canónico end-to-end: sincroniza gobierno, exporta DCAT-AP-ES y valida SHACL HVD.

6.5. Validación externa con el validador europeo (SEMIC/ITB)

Además de la validación SHACL local con las *shapes* DCAT-AP-ES congeladas, el catálogo exportado se ha sometido a una validación externa e independiente con el *SEMIC SHACL Validator* del *Interoperability Test Bed* (ITB) de la Comisión Europea [35]. Este servicio comprueba la conformidad de un grafo RDF frente a las especificaciones SEMIC oficiales. Se cargó el catálogo `validation_suite_catalog.jsonld` y se validó contra el grupo DCAT-AP, perfil **DCAT-AP HVD**, versión **3.0.0 Base Zero** (sin conocimiento de fondo), con sintaxis de entrada JSON-LD.

El resultado, recogido en la figura 6.1, fue **SUCCESS** con **0 errores, 0 advertencias y 0 mensajes**. Este hallazgo es relevante por dos motivos. Primero, confirma que la salida no solo cumple el perfil nacional DCAT-AP-ES validado localmente, sino también el perfil europeo *DCAT-AP HVD* verificado por una herramienta oficial ajena a la plataforma. Segundo, la modalidad *Base Zero* aplica las restricciones sin resolver vocabularios externos, de modo que una conformidad sin hallazgos evidencia que la estructura, las relaciones y las propiedades del grafo son correctas por sí mismas y no por inferencias del validador.



European Commission

SEMIC

Semantic Interoperability across Europe

SEMIC SHACL Validator

 GitHub

 SEMIC Support Centre

 SEMIC SHACL Validator

 SEMIC XML Validator

This service enables you to check whether your RDF graph or data model conforms to SEMIC specifications. For each specification, a set of rules is implemented to determine if the relevant SHACL conventions are followed. These rules define constraints on the structure, relationships, and properties within the models. Validation is also available via [REST](#) and [SOAP](#) APIs to facilitate integration and conformance testing.

The content to validate can be provided as a file, URI, or direct input. Documentation on using the [Interoperability Test Bed's \(ITB\) RDF validator](#) is available [here](#).

More information available on the [SEMIC Support Centre](#) regarding the SEMIC specifications. If you would like to stay informed about SEMIC specifications and related tool developments, you can join the mailing list [here](#).

File to validate

File

validation_suite_catalog.jsonld

Group

DCAT-AP

Validate for

DCAT-AP HVD

Release

3.0.0 Base Zero (no background knowledge)

Content syntax

JSON-LD

Validate

Validation result

Overview

Date:

2026-05-31T20:37:29.371Z

File name:

validation_suite_catalog.jsonld

Validation type:

DCAT-AP HVD 3.0.0 Base Zero validation (no background knowledge)

Findings:

0 error(s), 0 warning(s), 0 message(s)

Result:

SUCCESS

Download report

Download SHACL shapes

Download validated content

Figura 6.1: Validación externa del catálogo en el *SEMIC SHACL Validator* (ITB, Comisión Europea): perfil DCAT-AP HVD 3.0.0 Base Zero, resultado SUCCESS sin errores ni advertencias. Elaboración propia.

6.6. Resultados del caso de uso de validación

El caso de uso de validación produce dos datasets publicables y sus recursos asociados:

- *Agenda cultural pública*, a partir de `gold.agenda_cultural_publica`;
- *Resumen de movilidad por municipio*, a partir de `gold.movilidad_resumen_municipio`.

La documentación del repositorio registra una ejecución completa de la suite con los siguientes resultados: conformidad runtime, conformidad técnica, conformidad de gobierno, conformidad SHACL, idempotencia positiva, dos tablas exportadas, dos cambios aplicados en la primera ejecución y cero cambios aplicados en la segunda. Esta última cifra es relevante porque evidencia idempotencia: una vez sincronizado el estado, repetir el workflow no vuelve a aplicar operaciones innecesarias.

Comprobación	Resultado documentado	Interpretación
Estado runtime	Conforme	OpenMetadata contiene los activos esperados.
Gobierno aplicado	Conforme	La hoja funcional coincide con metadatos vivos.
SHACL HVD (local)	Conforme con warnings permitidos	No hay violaciones bloqueantes.
Validación europea (ITB)	SUCCESS (0/0/0)	Conforme con DCAT-AP HVD 3.0.0 en el validador oficial europeo.
Tablas exportadas	2	Se exportan los dos datasets <code>gold</code> .
Segunda ejecución	0 cambios aplicados	El workflow es idempotente.

Tabla 6.1: Resumen de resultados del caso de uso de validación. Elaboración propia a partir de la documentación del repositorio.

6.7. Artefactos generados

Los artefactos esperados son:

- catálogo JSON-LD exportado;
- informe SHACL en Turtle;

- informe runtime en JSON;
- plan de workflow en JSON;
- resumen de suite de validación;
- informe de validación legible en HTML y PDF.

A partir del resumen JSON de la validación, la plataforma genera un *informe legible* en HTML y PDF mediante el comando `om_dcat_sync render-report`, que se apoya en la biblioteca `fpdf2` para el PDF. El informe resalta los indicadores de conformidad (estado runtime, gobierno, idempotencia y SHACL) y aplanar el resto de resultados en un listado consultable. Puede generarse desde la consola web —pantalla *Validación*— o desde el CLI, y queda disponible como evidencia autocontenida para revisión, defensa o entrega a terceros sin necesidad de inspeccionar el JSON en bruto. La consola web sirve el informe HTML con su tipo **MIME!** (**MIME!**) real mediante el botón *Abrir*, de modo que se renderiza directamente en el navegador (figura 6.2).

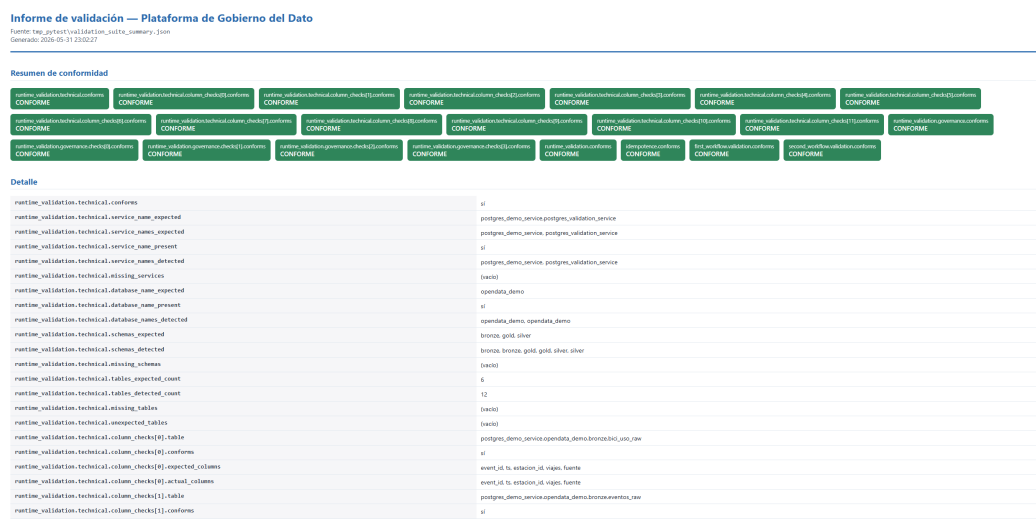


Figura 6.2: Informe de validación HTML renderizado en el navegador desde la consola web: resumen de conformidad por indicador y detalle del resultado. Elaboración propia.

Estos ficheros se generan en `tmp_pytest/`. No se versionan porque son reproducibles y dependen de la ejecución local, pero sí se referencian como evidencia regenerable. La cadena de pasos para regenerarlos desde un repositorio limpio se describe en el Anexo A, y el recorrido visual que conduce a cada artefacto en el Anexo F.

7. Discusión

7.1. Cumplimiento del problema planteado

La solución resuelve el problema principal: conectar un catálogo técnico en OpenMetadata con una salida interoperable DCAT-AP-ES validable. La plataforma no se limita a documentar un mapeo, sino que implementa un flujo ejecutable que descubre activos técnicos, permite curación funcional, aplica metadatos, exporta JSON-LD y valida el resultado. El Anexo F narra ese flujo de extremo a extremo desde la consola web, hasta dejar el archivo RDF listo para entregarlo a un harvester compatible.

La decisión de usar PostgreSQL de referencia en lugar de cosechar CKAN refuerza la defendibilidad del trabajo. CKAN habría permitido transformar metadatos ya publicados; PostgreSQL y OpenMetadata permiten demostrar gobierno desde activos técnicos, que es más coherente con las competencias de ingeniería, Big Data y cloud del máster.

7.2. Beneficios obtenidos

Los principales beneficios son:

- reproducibilidad del entorno y del caso de uso;
- separación entre metadatos técnicos y curación funcional;
- núcleo Python único para CLI, scripts y web;
- validación formal mediante SHACL;
- trazabilidad entre SQL, OpenMetadata, hoja funcional, JSON-LD e informe de validación;

-
- facilidad de operación mediante consola web.

7.3. Limitaciones

La solución tiene limitaciones deliberadas:

- una tabla SQL no equivale semánticamente a un `dcat:Dataset`; en el caso de uso se usa como aproximación técnica gobernable;
- se genera una única `Distribution` y un único `DataService` por dataset;
- la clasificación HVD es una hipótesis de validación dentro de la plataforma, no una calificación jurídica automática;
- la validación se centra en metadatos, no en profiling de contenido ni calidad fila a fila;
- no se publica automáticamente en `datos.gob.es`, CKAN ni `data.europa.eu`;
- el modelo OpenMetadata se mantiene mínimo y no cubre todos los campos opcionales de DCAT-AP-ES.

La primera limitación merece una precisión conceptual. Un `dcat:Dataset` es una abstracción editorial: una colección de datos publicada y mantenida por un agente como una unidad coherente, con independencia de cómo se almacene físicamente [2]. Una tabla SQL, en cambio, es una estructura física de almacenamiento. La correspondencia «una tabla es un dataset» es, por tanto, una decisión de modelado y no una equivalencia natural: a nivel técnico un dataset podría corresponder igualmente a una vista, a una vista materializada, al resultado de una consulta o a la unión lógica de varias tablas. Conceptos como un cursor o un índice no son candidatos válidos, porque un cursor es un mecanismo de recorrido y un índice una estructura de acceso, no un conjunto de datos publicable. En el caso de uso de validación se elige la tabla `gold` como unidad publicable por ser la representación más estable y autocontenida del dato refinado; en un escenario real, la hoja de gobierno permitiría asociar el dataset a la vista o agregación que mejor represente la unidad de publicación, sin alterar el núcleo.

Estas limitaciones no invalidan el trabajo: forman parte del alcance funcional declarado y delimitan la frontera entre lo que la plataforma demuestra de forma reproducible y lo que correspondería a una iniciativa de portal de datos abiertos a escala institucional, con presupuesto, equipo y compromisos de servicio diferenciados.

7.4. Amenazas a la validez

La principal amenaza a la validez externa es que el caso de uso trabaja con datos sintéticos y un número reducido de datasets publicables. La arquitectura, sin embargo, está diseñada para escalar a más tablas `gold` mediante la misma hoja funcional y el mismo workflow.

Conviene aclarar que la conformidad SHACL «con warnings permitidos» no constituye una amenaza a la validez. En SHACL, una violación con severidad `sh:Violation` indica el incumplimiento de una restricción obligatoria; un `sh:Warning`, en cambio, señala únicamente que el perfil *recomienda* cubrir propiedades adicionales —recomendadas u opcionales— que no son obligatorias [13]. El catálogo exportado no presenta violaciones bloqueantes: los warnings solo invitan a enriquecer metadatos no exigidos por el perfil. Aumentar esa cobertura mejora la calidad editorial del catálogo, pero su ausencia no invalida la conformidad obligatoria demostrada.

7.5. Posicionamiento frente a entornos Big Data

El enunciado oficial sitúa el trabajo en “entornos Big Data y de computación en la nube” [1], expresión que conviene matizar para evitar una lectura equívoca. La plataforma no opera en la capa de procesamiento masivo de datos, sino en la capa de gobierno de metadatos e interoperabilidad. El caso de uso de validación trabaja con un volumen reducido de datos sintéticos y, por tanto, no constituye Big Data en el sentido estricto de volumen, velocidad y variedad. Esta frontera es coherente con la distinción establecida en el capítulo 2: el objeto gobernado son los metadatos, no los datos de negocio.

La relevancia para entornos Big Data es, en consecuencia, indirecta pero sustantiva. Los ecosistemas de datos a gran escala se caracterizan por activos heterogéneos, distribuidos y de múltiples fuentes, cuya utilidad depende de que permanezcan descubribles, descritos y reutilizables. Esa función —catalogación, descripción semántica e interoperabilidad— es precisamente la que DAMA encuadra como gestión de metadatos dentro del gobierno del dato [5]. La plataforma demuestra dicho mecanismo de forma reproducible: descubre activos técnicos, los describe, los exporta como catálogo DCAT-AP-ES y valida su conformidad.

En un escenario Big Data real, el proyecto se utilizaría sin cambios en su lógica de negocio, dado que el flujo opera sobre metadatos y es independiente del volumen del dato subyacente. El crecimiento se absorbería por tres vías: (i) la incorporación de nuevas fuentes y servicios en OpenMetadata, catálogo concebido para inventarios técnicos amplios y

heterogéneos [14]; (ii) la ampliación del contrato funcional añadiendo más tablas `gold` a la misma hoja de gobierno y al mismo workflow idempotente, sin modificar el núcleo; y (iii) el despliegue nativo en la nube, que habilita el escalado de la capa de catálogo y gobierno conforme a las primitivas de orquestación de Kubernetes [20]. La salida interoperable en DCAT-AP-ES es, además, el vehículo por el que un patrimonio de datos extenso se hace federable y localizable entre portales nacionales y europeos [3, 2]. En síntesis, el trabajo no escala procesando más datos, sino gobernando más metadatos, que es la dimensión en la que el gobierno aporta valor a un entorno Big Data.

7.6. Gobierno supervisado y responsabilidad del operador

Aunque el flujo es ampliamente automatizable e idempotente, una parte esencial del gobierno reside en decisiones que deben permanecer bajo autoridad humana. Determinar qué activos son publicables, asignar la temática NTI-RISP [27], clasificar un dataset como de alto valor o redactar su título, descripción y publicador no son transformaciones deterministas, sino juicios editoriales y de responsabilidad. El carácter manual y dinámico de esta curación no es una carencia del trabajo, sino una propiedad deseada del gobierno del dato.

El marco DAMA es explícito en este punto: el gobierno del dato asigna responsabilidad (*accountability*) a roles definidos —propietario y *steward* del dato— y la automatización debe asistir esa función, no sustituirla [5, 4]. Por consiguiente, la persona que opera o gobierna la plataforma debe conocer y aplicar las buenas prácticas del dominio, y responder de aquello que publica. La interoperabilidad técnica no exime de la responsabilidad sobre el contenido publicado.

Este principio tiene una implicación normativa concreta. La condición de conjunto de datos de alto valor está determinada por el Reglamento de Ejecución (UE) 2023/138 [34]; la plataforma trata la clasificación HVD como hipótesis de validación y no como una calificación jurídica automática, como ya se reconoció entre las limitaciones de este capítulo. Por ello, la categorización debe ser revisada y autorizada por una persona responsable antes de cualquier publicación efectiva, y nunca derivarse de forma ciega por mera configuración.

El diseño de la plataforma materializa esta filosofía mediante salvaguardas explícitas. La asistencia de autorrellenado solo completa campos vacíos con valores de validación y nunca sobrescribe la curación introducida por el operador, de modo que la automatización ayuda sin desplazar el criterio humano. Del mismo modo, las líneas de trabajo futuro orientadas a la ejecución periódica mediante un planificador o CI/CD, que se recogen a continuación, deben entenderse como automatización de la reproducibilidad, no como delegación de la

decisión de publicar. La recomendación operativa es inequívoca: que el sistema sea técnicamente capaz de automatizar el flujo de extremo a extremo no implica que deba ejecutarse sin supervisión; la publicación de metadatos gobernados, por su visibilidad externa y por sus consecuencias jurídicas y reputacionales, debe ser un acto autorizado y supervisado.

7.7. Trabajo futuro

Las líneas de evolución más razonables son:

- publicar automáticamente el JSON-LD en un CKAN o portal compatible cuando exista un destino real;
- ampliar la cobertura de metadatos recomendados y opcionales de DCAT-AP-ES;
- incorporar aprobaciones editoriales y roles si la web pasa a uso multiusuario;
- automatizar ejecuciones periódicas mediante Airflow, CI/CD o un scheduler;
- validar el comportamiento con más fuentes técnicas y más datasets *gold*, por ejemplo mediante ingesta desde APIs o bases de datos públicas.

8. Conclusiones

El TFM ha diseñado e implementado una Plataforma de Gobierno del Dato que permite gobernar metadatos técnicos en OpenMetadata y exportarlos como catálogo DCAT-AP-ES validable. El trabajo cumple el objetivo general porque no se queda en una configuración conceptual: proporciona infraestructura reproducible, flujo de ingesta, curación funcional, sincronización de gobierno, exportación JSON-LD y validación SHACL.

Los objetivos específicos también se han cubierto. Se ha analizado DCAT-AP-ES y su relación con DCAT-AP y HVD; se ha definido un mapeo entre OpenMetadata y las clases activas del perfil; se han configurado custom properties y tags; se ha implementado un workflow idempotente; se ha generado una salida interoperable; y se ha validado el resultado mediante pruebas, validación runtime y SHACL.

La aportación más relevante es la separación entre tres responsabilidades: OpenMetadata como catálogo técnico, una hoja funcional como punto de curación editorial y un núcleo Python como mecanismo de sincronización, exportación y validación. Esta separación permite que la consola web opere la plataforma sin convertirse en una segunda fuente de reglas de negocio.

El cambio de CKAN a PostgreSQL de referencia se considera una decisión metodológica adecuada. Permite demostrar gobierno de metadatos desde activos técnicos reales y reproducibles, en lugar de cosechar metadatos ya publicados por otro catálogo. CKAN mantiene sentido como posible destino o federación futura, pero no como origen canónico del caso de uso de validación.

El trabajo presenta limitaciones explícitas: opera sobre datos sintéticos, cubre un subconjunto controlado de DCAT-AP-ES, no publica automáticamente en portales externos y no evalúa calidad del dato de negocio. Estas limitaciones se han diseñado conscientemente como frontera de alcance, no como déficit técnico, y no afectan al resultado principal: una

solución reproducible, trazable y defendible que transforma activos técnicos gobernados en un catálogo interoperable validado formalmente.

A. Reproducción del caso de uso de validación

Este anexo concentra los comandos exactos para reproducir el caso de uso de validación desde un repositorio limpio. Se asume Windows 10 u 11 con PowerShell, Docker Desktop, Node.js 20 o superior, Python 3.11 o superior y MiKTeX para compilar la memoria. Los comandos están extraídos de `scripts/infra/` y de la documentación canónica del repositorio.

A.1. Preparación del entorno

```
1 python -m pip install --upgrade pip
2 python -m pip install -e .\tfm_ingestor
```

Código A.1: Instalación de dependencias Python del núcleo `om_dcat_sync`.

```
1 cd .\web
2 pnpm install
```

Código A.2: Instalación de dependencias de la consola web con pnpm.

A.2. Despliegue de infraestructura

```
1 powershell -ExecutionPolicy Bypass -File .\scripts\infra\launch_infra.ps1
```

```
2 powershell -ExecutionPolicy Bypass -File .\scripts\infra\ingest_postgres_double.  
  ↪ ps1
```

Código A.3: Lanzamiento de Kind, Helm e ingesta doble.

A.3. Ejecución del workflow canónico

```
1 python -m om_dcat_sync workflow run --dry-run  
2 python -m om_dcat_sync workflow run --allow-warnings
```

Código A.4: Ejecución del workflow canónico end-to-end.

A.4. Suite reproducible

```
1 powershell -ExecutionPolicy Bypass -File .\scripts\infra\run_validation_suite.ps1  
  ↪
```

Código A.5: Suite reproducible documentada.

La salida de la suite incluye `runtime_validation_report.json`, `dcat_catalog.jsonld` y `dcat_validation_report.ttl` en `tmp_pytest/`.

B. Detalle de infraestructura

Este anexo amplía la sección 5.3 con los detalles de despliegue que conviene tener accesibles sin entrar al repositorio.

B.1. Versión de OpenMetadata

El despliegue de referencia del caso de uso de validación utiliza **OpenMetadata 1.12.9**, instalado mediante los *charts* oficiales de Helm [36]. La versión exacta puede consultarse en cualquier momento contra la instancia viva mediante el endpoint `/api/v1/system/version` de la API REST. Las dependencias gestionadas por el *chart* `openmetadata-dependencies` son MySQL como almacén de metadatos y OpenSearch como motor de búsqueda. Las entidades de la API empleadas por la plataforma (servicios, bases de datos, esquemas, tablas, columnas, *custom properties* y *tags*) son estables en la rama 1.12.x, por lo que la lógica de gobierno no depende de detalles internos de una versión concreta.

B.2. Matriz de versiones del sistema

La reproducibilidad exige que todas las versiones del sistema estén fijadas y documentadas. La tabla B.1 recoge las versiones empleadas en el caso de uso de validación y el punto del repositorio donde quedan ancladas. El detalle operativo se mantiene actualizado en `docs/versiones.md`.

Componente	Versión	Dónde se fija
OpenMetadata (app y charts)	1.12.9	--version en launch_infra.ps1
Helm (CLI)	3.14.4	.tools/helm-v3.14.4/
Kind	0.31.0	prerrequisito (check_prereqs.ps1)
kubectl	1.34.1	prerrequisito
Docker Engine	28.5.1	prerrequisito
Python	3.12 (>= 3.10)	tfm_ingestor/pyproject.toml
Node.js	22.11	prerrequisito de web/
pnpm	11.x	gestor Node canónico
Next.js / React / TypeScript	16.2 / 19.2 / 6.0	web/package.json
pyshacl	>= 0.28	extra validation (pyproject.toml)
PyJWT / cryptography	>= 2.8 / 42	extra infra (pyproject.toml)
fpdf2	>= 2.7	extra report (pyproject.toml)
Bundle SHACL DCAT-AP-ES	1.0.0 (f2c8a888)	resources/shacl/manifest.json
Perfiles DCAT	DCAT-AP-ES 1.0.0 DCAT-AP 2.1.1 HVD 2.2.0	documentación oficial [3]

Tabla B.1: Matriz de versiones fijadas del sistema. Elaboración propia.

B.3. Valores Helm de OpenMetadata

Los valores canónicos viven en `k8s/openmetadata.values.yaml` y `k8s/openmetadata-dependencies.values.yaml`. Documentan: tamaño de pods, configuración de Elasticsearch (OpenSearch), credenciales por defecto del JWT de servicio, namespace y puertos expuestos.

B.4. Manifiesto de PostgreSQL de referencia

El manifiesto `k8s/postgres-demo.yaml` crea un Deployment, un Service y un ConfigMap con el contenido de `sql/opendata_demo_init.sql`. La inicialización es automática: el contenedor monta el SQL como volumen y lo ejecuta en el arranque.

B.5. Diagrama de despliegue

El diagrama de la figura 3.1 resume el despliegue. En operación real, el `kubectl port-forward` expone OpenMetadata en `localhost:8585` y PostgreSQL en `localhost:5432`.

B.6. Notas operativas

- El clúster Kind se borra con `kind delete cluster --name tfm` si se requiere partir de cero.
- Las credenciales por defecto son las de la documentación oficial de OpenMetadata [14]; en producción deberían rotarse.
- No se han activado *políticas de red* ni RBAC avanzado por estar fuera del alcance académico. Las *políticas de red* (recurso `NetworkPolicy` de Kubernetes) restringen qué *pods* pueden comunicarse entre sí y hacia el exterior; el RBAC (*Role-Based Access Control*) limita qué acciones puede ejecutar cada usuario o cuenta de servicio sobre los recursos del clúster. Ambos son controles de *hardening* habituales en producción que no aportan valor al caso de uso de validación, centrado en gobierno de metadatos y reproducibilidad.

B.7. Instalación portable y organismos con OpenMetadata propio

La plataforma es portable y no exige desplegar su propio OpenMetadata. El despliegue con Kind, Helm y PostgreSQL de referencia descrito en este anexo solo construye un entorno autocontenido y reproducible para el caso de uso de validación. La lógica de gobierno reside íntegramente en el núcleo Python `om_dcat_sync`, que únicamente necesita acceso a la API REST de un OpenMetadata y un token JWT válido.

En un organismo o empresa que ya disponga de OpenMetadata, la integración no requiere recrear infraestructura: basta con apuntar la plataforma a la instancia existente.

- Configurar `OPENMETADATA_BASE_URL` hacia la API del OpenMetadata del cliente (por ejemplo, `https://catalogo.organismo.es/api/v1`).
- Proporcionar un token de servicio en `OPENMETADATA_TOKEN` o `OPENMETADATA_JWT_TOKEN`, obtenido de un *bot* de ingesta del propio OpenMetadata.

-
- Omitir los pasos de creación de clúster (`launch_infra.ps1`) y ejecutar directamente el workflow de gobierno, exportación y validación, ya sea desde el CLI o desde la consola web.

De este modo, el mismo flujo —descubrimiento, curación funcional, sincronización idempotente, exportación DCAT-AP-ES y validación SHACL— opera sobre el catálogo técnico que el cliente ya mantiene, sin imponer su motor de despliegue ni duplicar su gobierno. La compatibilidad se ha verificado contra OpenMetadata 1.12.x (sección B.1); versiones próximas que conserven las mismas entidades de la API REST son igualmente compatibles. Esta separación entre lógica portable y despliegue de referencia es la que permite ofrecer la plataforma como capa de conformidad DCAT-AP-ES sobre un OpenMetadata preexistente.

C. Esquema de gobierno funcional

C.1. Estructura de la hoja `gold_governance.csv`

La hoja `tfm_ingestor/config/gold_governance.csv` usa separador `;` y las siguientes columnas:

- `publicar`: si o no; controla la inclusión en el catálogo exportado.
- `schema_name`: esquema PostgreSQL (siempre `gold` en el caso de uso de validación).
- `table_name`: nombre de tabla en PostgreSQL.
- `table_fqn`: identificador completo en OM (`service.database.schema.table`).
- `titulo_dataset`: título funcional del dataset.
- `descripcion_dataset`: descripción de la fila desde la perspectiva editorial.
- `publicador`: nombre del organismo publicador (UCLM en el caso de uso).
- `tematica_dcat`: tema NTI-RISP del dataset (`cultura_ocio`, `transporte`, etc.).
- `categoria_hvd`: categoría HVD del Reglamento (UE) 2023/138.
- `access_url_distribucion`: URL pública o estable de la distribución.

C.2. Defaults globales

El archivo `tfm_ingestor/config/governance_defaults.yaml` centraliza los valores constantes del catálogo. Su edición altera el comportamiento de todos los datasets simultáneamente, principio clave de la sección 3.7.1.

D. Mapeo DCAT-AP-ES y entregables

Este anexo resume el mapeo entre el catálogo técnico de OM y el perfil DCAT-AP-ES, y enumera los entregables del caso de uso de validación. La referencia completa y canónica del mapeo, con la matriz íntegra de propiedades, cardinalidades y elevaciones de obligatoriedad respecto a DCAT-AP, se mantiene en `docs/dcat_mapping.md`.

D.1. Correspondencia OpenMetadata → DCAT-AP-ES

El exportador traduce un subconjunto deliberadamente mínimo de elementos de OM a clases y propiedades DCAT-AP-ES, completando el resto por configuración (tabla D.1).

Elemento en OpenMetadata	Destino en DCAT-AP-ES
Table/View (capa gold)	<code>dcat:Dataset</code>
<i>custom property</i> <code>dcat_access_url</code>	<code>dcat:Distribution / dcat:accessURL</code>
<i>custom property</i> <code>dcat_hvd_category</code>	<code>dcatap:hvdCategory</code>
<i>custom property</i> <code>dcat_publisher_name</code>	<code>foaf:name</code> del publicador
<i>tags</i> <code>dcat_theme.*</code>	<code>dcat:theme</code>
<i>defaults</i> del catálogo (YAML)	<code>dcat:Catalog</code>
<i>defaults</i> HVD (YAML)	<code>dcat:DataService</code> y restricciones HVD derivadas

Tabla D.1: Correspondencia mínima OpenMetadata → DCAT-AP-ES. Elaboración propia a partir de `docs/dcat_mapping.md`.

D.2. Qué gobierna OpenMetadata y qué deriva el sistema

En OM solo se mantienen los metadatos personalizados imprescindibles para no alargar artificialmente el modelo: `displayName`, `description`, `dcat_publisher_name`, `dcat_hvd_category`, `dcat_access_url` y los *tags* `dcat_theme.*`. El resto del perfil activo se deriva durante la exportación: el `dcat:Catalog` completo desde `governance_defaults.yaml`; la legislación aplicable, las licencias HVD y los derechos de acceso; el `dcat:DataService` con su punto de acceso, descripción, contacto y página de documentación; y los enlaces `dcat:accessService` y `dcat:servesDataset` entre distribución, servicio y dataset.

D.3. Qué aporta activar el caso HVD

Activar el caso HVD no se limita a añadir una clasificación. En el perfil activo implica `dcatap:hvdCategory` en dataset y servicio, `dcatap:applicableLegislation` derivada para dataset, distribución y servicio, `dct:license` en distribución y servicio, un `dcat:DataService` explícito, `dcat:contactPoint` y `dct:accessRights` derivados de la configuración global, y la validación SHACL específica del caso `hvd`.

D.4. Entregables del caso de uso

El flujo produce un conjunto cerrado de entregables reproducibles, todos regenerables en `tmp_pytest/` y visualizados a lo largo de la memoria (tabla D.2).

Entregable	Formato	Dónde se visualiza
Catálogo gobernado	JSON-LD	Listados 3.2 y 5.4; pantalla <i>DCAT</i> (fig. F.6)
Informe de validación SHACL	Turtle	Listado 3.3; pantalla <i>SHACL</i> (fig. F.8)
Informe del estado vivo	JSON	Sección 6, validación <i>runtime</i>
Resumen de la suite	JSON	Tabla 6.1; pantalla <i>Validación</i> (fig. F.7)
Informe legible de validación	HTML y PDF	Generado por <code>render-report</code> ; pantalla <i>Artefactos</i> (fig. F.9)

Tabla D.2: Entregables del caso de uso de validación y su visualización en la memoria. Elaboración propia.

E. Validación y consola web

E.1. Comandos de validación

```
1 python -m om_dcat_sync validate-runtime --strict `
2     --output tmp_pytest/runtime_validation_report.json
3 python -m om_dcat_sync validate-dcat --profile-case hvd --allow-warnings `
4     --report-output tmp_pytest/dcat_validation_report.ttl
```

Código E.1: Validación independiente y por suite.

E.2. Estructura de la consola web

La consola web en `web/` es una aplicación Next.js con rutas independientes por etapa. El listado 5.5 muestra cómo persisten los jobs. Documentación operativa completa: `docs/app_web.md`.

E.3. Pruebas de la consola web

```
1 cd .\web
2 pnpm test
```

Código E.2: Pruebas Vitest de la consola web.

F. Demostración guiada de la plataforma

Este anexo describe el recorrido completo que sigue un operario para gobernar metadatos en la plataforma desde el navegador, sin invocar comandos directamente sobre el núcleo Python. Su objetivo es triple: servir de referencia para reproducir la solución, soportar una defensa visual del trabajo y dejar trazado qué artefacto se obtiene en cada paso, hasta el archivo final RDF que un harvester externo puede consumir.

F.1. Propósito y supuestos iniciales

La demostración asume que el entorno se ha preparado siguiendo el Anexo A: el clúster Kind está activo con OM y sus dependencias desplegados mediante Helm, el PostgreSQL de referencia está inicializado con los esquemas *bronze*, *silver* y *gold*, los dos servicios de ingesta están registrados en OM y la consola web se sirve localmente con `pnpm dev` desde `web/`. La verificación rápida de prerequisites se realiza con el listado F.1.

```
1 kubectl get pods                                # OpenMetadata, MySQL, OpenSearch,  
    ↪ Postgres  
2 curl http://localhost:8585/api/v1/health-check  
3 cd .\web; pnpm dev                               # consola web en http://localhost:3000
```

Código F.1: Comprobación rápida del entorno antes de iniciar la demostración.

A partir de aquí toda la operación discurre por el navegador. La consola web no implementa lógica de gobierno: invoca los mismos servicios Python que el CLI (sección 5.5) y los scripts de `scripts/infra/`. La equivalencia entre acción visual y comando subyacente queda recogida en la tabla resumen al cierre del anexo (tabla F.1).

F.2. Apertura de la consola

La página inicial muestra el resumen de la plataforma y un *sidebar* con las once secciones de operación. El orden recomendado de uso reproduce el flujo lógico: *Infraestructura* → *Ingesta* → *Gobierno* → *Workflow* → *DCAT* → *Validación* → *SHACL* → *Artefactos*. Las dos secciones complementarias, *Estado vivo* y *Ejecuciones*, permiten consultar respectivamente la situación actual de OM y el historial de *jobs* versionados.

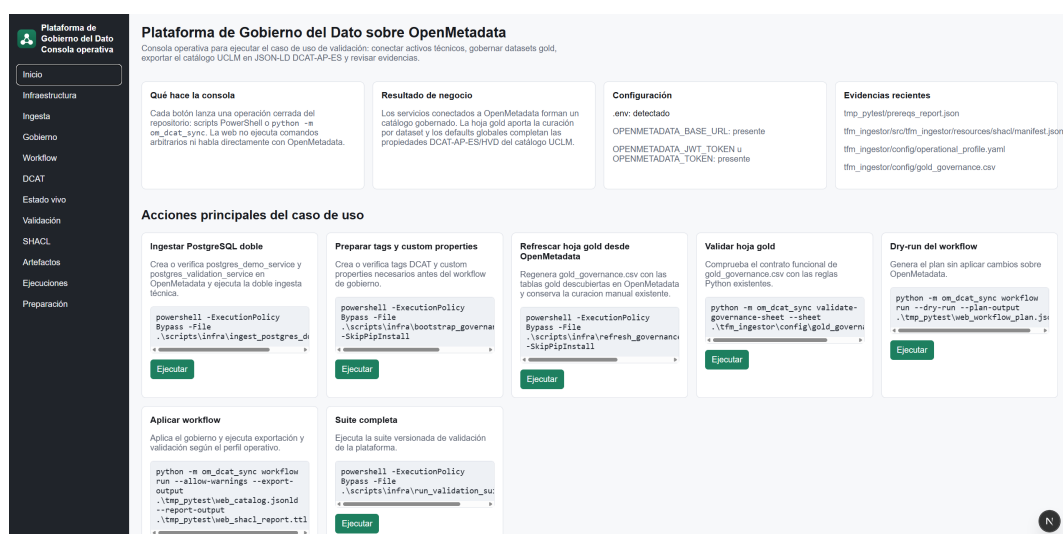


Figura F.1: Pantalla inicial de la consola web y *sidebar* con las secciones operativas. Elaboración propia.

F.3. Estado de la infraestructura

La pantalla *Infraestructura* expone dos acciones: comprobar prerequisites y, opcionalmente, ejecutar un reset limpio. La comprobación lanza un *job* que verifica la presencia de los binarios (*docker*, *kubectl*, *helm*, *kind*) y la accesibilidad de los servicios desplegados. Tras unos segundos se muestra el resultado con código de salida, log completo y duración. En la demostración solo se ejecuta la comprobación, ya que el entorno está listo.

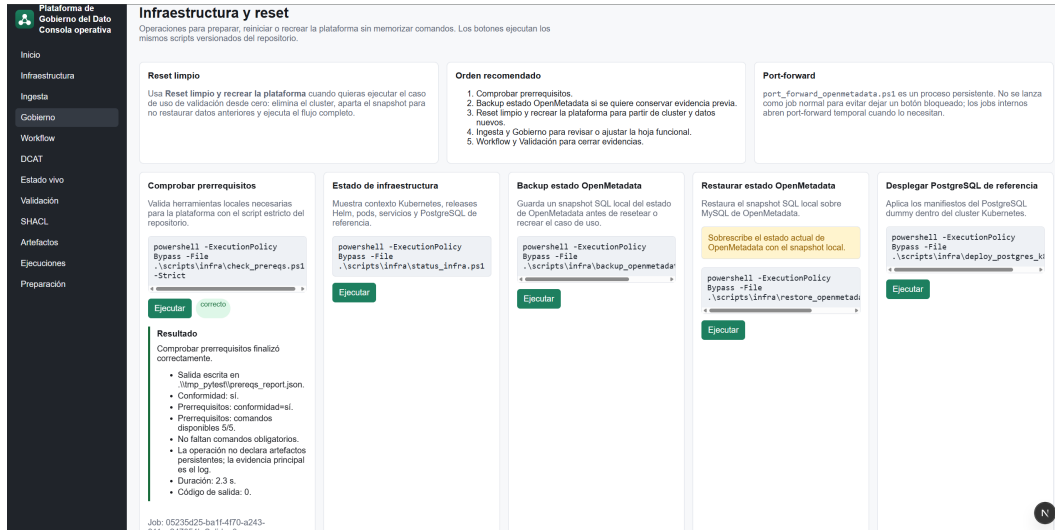


Figura F.2: Pantalla *Infraestructura*: comprobación de prerequisites y opciones de reset limpio. Elaboración propia.

F.4. Servicios y datasets ingeridos

En la pantalla *Ingesta* se observa el resultado de la doble ingesta técnica. Aparecen los dos DatabaseService en OM, postgres_demo_service y postgres_validation_service, cada uno con la misma estructura interna: la base de datos opendata_demo, los esquemas bronze, silver y gold, y las tablas descubiertas en cada esquema. La demostración confirma que las dos tablas gold se han registrado en ambos servicios y, por tanto, generan cuatro datasets publicables identificados por su FQN.

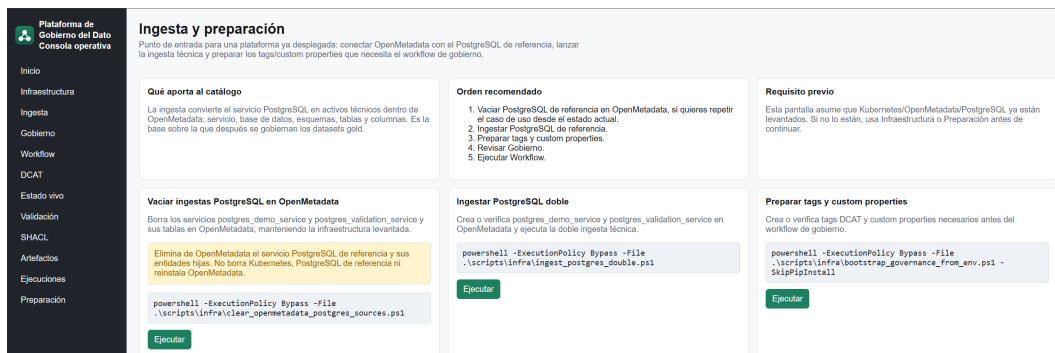


Figura F.3: Pantalla *Ingesta*: servicios PostgreSQL y datasets gold descubiertos por OpenMetadata. Elaboración propia.

F.5. Curación funcional en la pantalla *Gobierno*

Esta pantalla es el centro del recorrido. Concentra toda la curación funcional sobre la hoja `gold_governance.csv` (tabla 4.3) y se utiliza en cuatro pasos.

1. Refrescar hoja desde OpenMetadata. La acción descubre las tablas `gold` de los dos servicios y añade las filas correspondientes a la hoja, preservando la curación previa. El operario ve cómo aparece la lista de datasets con su `table_fqn` único.

2. Editar metadatos DCAT-AP-ES. Por cada dataset publicable se completa título funcional, descripción, publicador, temática NTI-RISP, categoría HVD y Uniform Resource Locator (URL) de la distribución. La pantalla ofrece listas desplegables para las taxonomías controladas y validación inmediata de los campos obligatorios.

3. Validar la hoja. Una segunda acción ejecuta la validación funcional y marca cualquier fila incompleta o inconsistente; el operario puede corregir y volver a validar antes de aplicar nada en OM.

4. Guardar. El guardado persiste la hoja CSV en disco. Hasta este punto no se ha modificado el estado de OM: solo se han preparado los datos editoriales que el *workflow* aplicará en el siguiente paso.

Plataforma de Gobierno del Dato
Consola operativa

Gobierno funcional gold
Esta es la interfaz funcional de los datasets `gold` concretos de la plataforma. La hoja cubre los campos por dataset que decide una persona responsable del catálogo; los demás obligatorios HVD se derivan desde la configuración global y el exportador DCAT.

Catálogo UCLM
governance_default.yml: fija el organismo publicador, URL institucional, licencias, contacto, legislación HVD y URL base. La hoja solo recoge lo que cambia por dataset.

Quié significa publicar
si significa que la tabla `gold` entra en el catálogo DCAT exportado y debe tener título, descripción, publicador, temática, categoría HVD y URL de acceso, no deja la tabla fuera del contrato publicable de la plataforma.

Cobertura HVD
La hoja no contiene todo DCAT-AP-ES HVD. Contiene la curación funcional por dataset. Catálogo, legislación, licencias, DataService, contactPoint y páginas de documentación se generan desde `governance_default.yml` y `dcats-export.yml`.

Listas controladas
Iswat i.e. it uses the sectors NTI-RISP+ completadas. categoria_hvd usa las seis categorías europeas HVD.

Hoja gold
Los identificadores técnicos son solo lectura. La validación autoritativa la hace Python; la web bloquea los errores de vocabulario y formato más frecuentes antes de guardar.

publicar	schema_name	table_name	table_fqn	título_dataset	descripcion_dataset	publicador	tematica_dcat	categoria_hvd	accesos_url
si - publicar en el catá	gold	agenda_cultural_publica	postgres_demo_service.opendata_demo.gold:agenda_cultural_publica	Agenda cultural pública	Relación de eventos culturales públicos con fecha, municipio, categoría y asistencia estimada (generada desde el ...)	UCLM	Cultura y ocio	Estadística	...
si - publicar en el catá	gold	movilidad_resumen_municipio	postgres_demo_service.opendata_demo.gold:movilidad_resumen_municipio	Resumen de movilidad	Resumen diario de viajes agregados por municipio para ...	UCLM	Transporte	Movilidad	...
si - publicar en el catá	gold	agenda_cultural_publica	postgres_validation_service.opendata_demo.gold:agenda_cultural_publica	Agenda cultural pública	Relación de eventos culturales públicos (generada por segunda ...)	UCLM	Cultura y ocio	Estadística	...
si - publicar en el catá	gold	movilidad_resumen_municipio	postgres_validation_service.opendata_demo.gold:movilidad_resumen_municipio	Resumen de movilidad	Resumen diario de viajes agregados por municipio (generado ...)	UCLM	Transporte	Movilidad	...

Autorellenar vacíos **Guardar hoja** **Recargar**

Refrescar hoja gold desde OpenMetadata **Validar hoja gold**

Figura F.4: Pantalla *Gobierno*: edición de la hoja `gold_governance.csv` con listas controladas para temática y categoría HVD. Elaboración propia.

F.6. Aplicación del workflow

La pantalla *Workflow* ofrece dos botones equivalentes al listado 6.4. La demostración los ejecuta en secuencia para mostrar el comportamiento idempotente.

Dry-run. El primer botón ejecuta `workflow run --dry-run`. El núcleo Python descubre las tablas, fusiona los *defaults* YAML con la hoja editada y genera un plan en JSON con la lista exacta de operaciones que se aplicarían sobre OM (PATCH, POST, etiquetas a añadir). El operario revisa el plan sin que nada haya cambiado todavía.

Apply. El segundo botón ejecuta `workflow run --allow-warnings` y aplica el plan. Al completarse, el resumen muestra el número de cambios realmente aplicados y los artefactos producidos: el catálogo JSON-LD y el informe SHACL.

Idempotencia. Repetir el botón *Apply* una segunda vez es ilustrativo: el resumen indica *0 cambios aplicados*, lo que evidencia visualmente la propiedad de idempotencia que el capítulo 6 valida formalmente.

Workflow
Ejecuta el dry-run y aplica el gobierno usando el comando canónico del repo. Es el paso que conecta hoja funcional, defaults globales, OpenMetadata, exportación DICAT y validación SHACL.

Dry-run
Genera un plan sin aplicar cambios. Sirve para revisar qué datasets se gobernarán y qué metadatos se sincronizarán antes de tocar OpenMetadata.

Dry-run del workflow
Genera el plan sin aplicar cambios sobre OpenMetadata.

`python -m oe_dcat_sync workflow run --dry-run --plan-output .\tmp_pytestweb_workflow_plan.json`

Ejecutar **Cancelar**

Resultado
Dry-run del workflow finalizó correctamente.

- Workflow: dry-run:si.
- Workflow: tablas descubiertas 6.
- Workflow: filas de hoja cargadas 2.
- Workflow: hoja funcional validada:si.
- Sincronización: cambios planificados 2.
- Sincronización: cambios aplicados 0.
- Sincronización: dry-run:si.
- Artefactos generados: 1/1.
- Duración: 0.8 s.
- Código de salida: 0.

tmp_pytestweb_workflow_plan.json JSON · 1094 bytes
Ver en artefactos

```
{
  "dry_run": true,
  "imports": 2,
  "plan": {
    "tableId": "postgres_demo.service.opendata_demo.gold.agenda_cultural_publica",
    "tableId": "9a918795-5eeb-4d18-83d4-49c968013118",
    "ops": [
      {
        "op": "replace",
        "path": "/description",
        "value": "Realización de eventos culturales públicos con fecha, municipio, categoría y asistencia estimada ingerida desde el servicio operativo PostgresQL."
      }
    ]
  }
}
```

Aplicar workflow
Aplica cambios idempotentes en OpenMetadata, exporta el catálogo UCLM en JSON-LD y valida el resultado contra las SHACL locales.

Aplicar workflow
Aplica el gobierno y ejecuta exportación y validación según el perfil operativo.

`python -m oe_dcat_sync workflow run --allow-warnings --export-output .\tmp_pytestweb_catalog.jsonld --report-output .\tmp_pytestweb_shacl_report.html`

Ejecutar **Cancelar**

Resultado
Aplicar workflow finalizó correctamente.

- Workflow: dry-run:si.
- Workflow: tablas descubiertas 6.
- Workflow: filas de hoja cargadas 2.
- Workflow: hoja funcional validada:si.
- Sincronización: cambios planificados 2.
- Sincronización: cambios aplicados 2.
- Sincronización: dry-run:si.
- Validación SHACL: conformidad:si.
- Validación SHACL: violaciones 0.
- Validación SHACL: warnings 22.
- Validación SHACL: tablas exportadas 2.
- Validación SHACL: datasets en catálogo 2.
- Artefactos generados: 2/2.
- Duración: 2.6 s.
- Código de salida: 0.

tmp_pytestweb_catalog.jsonld JSON-LD · 11485 bytes
Ver en artefactos

```
{
  "@context": {
    "dcat": "http://www.w3.org/ns/dcat",
    "dcap": "http://data.europa.eu/cap",
    "dct": "http://purl.org/dc/terms",
    "foaf": "http://xmlns.com/foaf/0.1/"
  }
}
```

Figura F.5: Pantalla *Workflow*: dry-run y aplicación con resumen de cambios. Elaboración propia.

F.7. Exportación DCAT-AP-ES

La pantalla *DCAT* dispara la exportación independiente del catálogo. Aunque el *workflow* ya produce el mismo artefacto, esta pantalla permite regenerar el JSON-LD sin reaplicar el gobierno: útil si se ajusta únicamente algún *default* global. El resultado es el archivo `tmp_pytest/dcat_catalog.jsonld` con las cinco clases DCAT activas (Catalog, Dataset, Distribution, DataService, Agent).

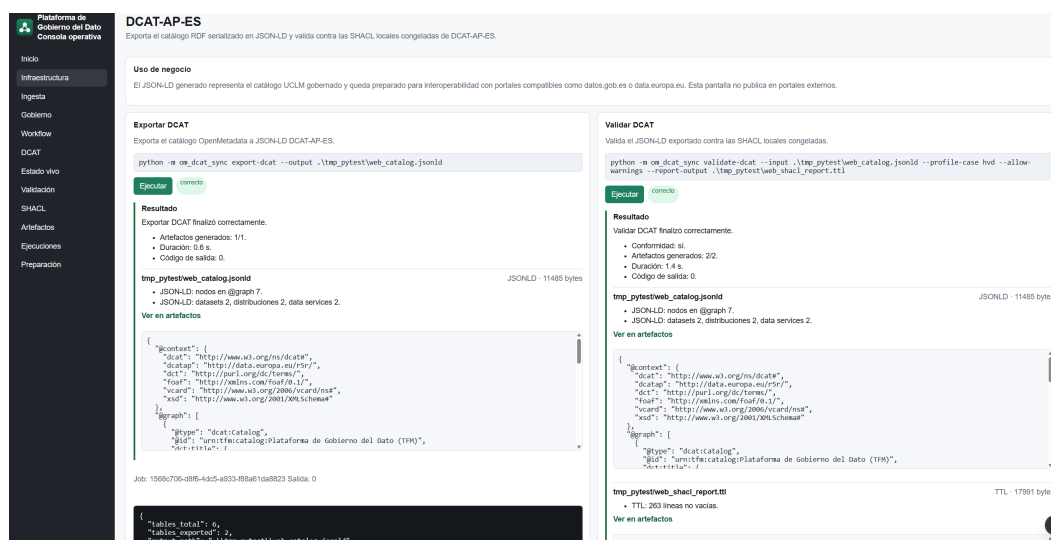


Figura F.6: Pantalla *DCAT*: exportación del catálogo JSON-LD y previsualización. Elaboración propia.

F.8. Validación SHACL y suite reproducible

La pantalla *Validación* ejecuta la suite reproducible completa (`run_validation_suite.ps1`): comprueba el estado vivo de OM, lanza el *workflow* dos veces para verificar la idempotencia y valida el catálogo exportado contra las *shapes* SHACL HVD vendorizadas. El resumen final consolida los códigos de salida de cada etapa en un único informe JSON.

La pantalla *SHACL*, complementaria, muestra el *manifiesto de las shapes DCAT-AP-ES congeladas* en el repositorio (`resources/shacl/manifest.json`): origen, versión y *commit* de las *shapes*. Su función es evidenciar que la validación no descarga *shapes* en tiempo de ejecución, sino que usa exclusivamente las locales. El informe SHACL propiamente dicho —el Turtle con severidades (`sh:Violation`, `sh:Warning`, `sh:Info`), nodo afectado y mensaje—

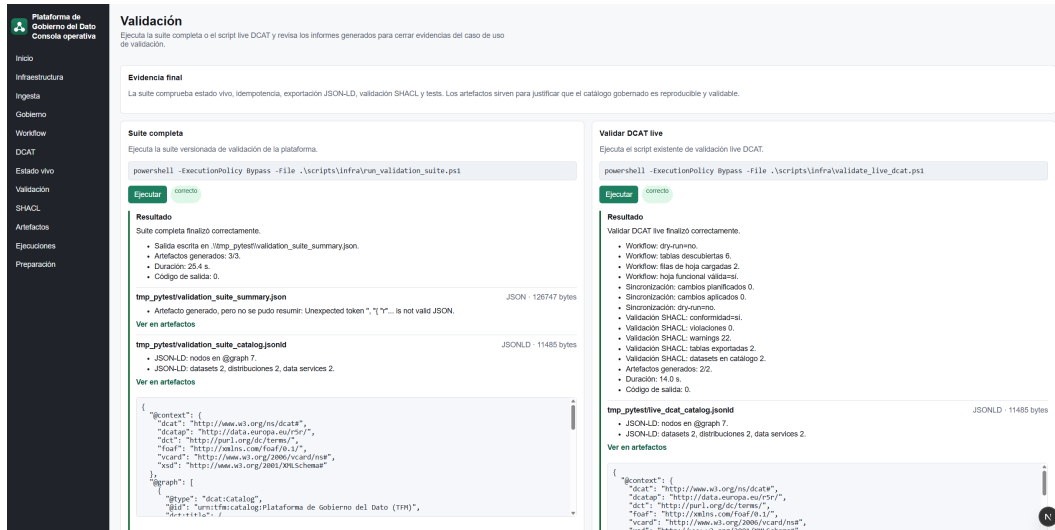


Figura F.7: Pantalla *Validación*: ejecución de la suite reproducible y resumen consolidado. Elaboración propia.

se genera como artefacto `*_shacl_report.ttl` y se consulta desde la pantalla *Artefactos*; en la demostración no presenta violaciones bloqueantes, en línea con el capítulo 6.

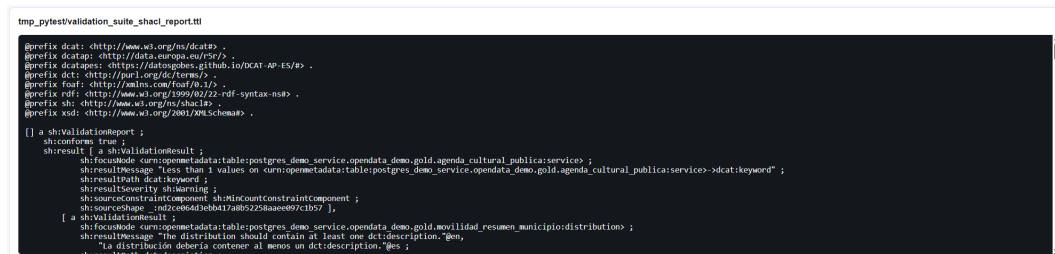


Figura F.8: Pantalla *SHACL*: manifiesto de las *shapes* DCAT-AP-ES congeladas (sin descarga en tiempo de ejecución). Elaboración propia.

F.9. Artefactos y descarga del catálogo RDF

La pantalla *Artefactos* expone todos los ficheros generados en `tmp_pytest/` con dos acciones por fila: *Ver* (previsualización de texto en la misma página) y *Abrir* (sirve el fichero con su tipo **MIME!** real en una pestaña nueva, de modo que el HTML se renderiza como página y el PDF se abre en el visor del navegador). El operario puede abrir o descargar directamente `dcat_catalog.jsonld` —el artefacto final del recorrido—, el informe SHACL en Turtle o el

informe HTML/PDF de validación.

The screenshot shows the 'Artefactos' (Artifacts) section of a web application. On the left is a dark sidebar with a menu containing: Inicio, Infraestructura, Ingesta, Gobierno, Workflow, DCAT, Estado vivo, Validación, SHACL, Artefactos (highlighted), Ejecuciones, and Preparación. The main area has a title 'Artefactos' and a subtitle 'Ficheros reproducibles generados por los comandos y scripts de la plataforma.' Below this is a table with columns: Ruta, Estado, Tamaño, Actualizado, and Acción. The table lists 18 artifacts, including JSON, HTML, PDF, and TII files, with their status (exists or not), size, and last update time. Each row has a 'Ver' button in the 'Acción' column. One row, 'tmp_pytest/validation_suite_catalog.json', has a blue link icon in the 'Ruta' column. A small 'N' icon is in the bottom right corner of the table area.

Ruta	Estado	Tamaño	Actualizado	Acción
tmp_pytest/prereqs_report.json	existe	1973	2026-05-31T20:14:54.301Z	Ver
tmp_pytest/validation_suite_summary.json	existe	126747	2026-05-31T20:20:19.706Z	Ver
tmp_pytest/validation_report.html	no existe	0	-	Ver
tmp_pytest/validation_report.pdf	no existe	0	-	Ver
tmp_pytest/runtime_validation_report.json	existe	8562	2026-05-31T20:20:12.807Z	Ver
tmp_pytest/validation_suite_catalog.json	existe	11485	2026-05-31T20:20:13.504Z	Ver
tmp_pytest/validation_suite_shacl_report.tii	existe	17932	2026-05-31T20:20:14.418Z	Ver
tmp_pytest/web_catalog.jsonld	existe	11485	2026-05-31T20:18:24.853Z	Ver
tmp_pytest/web_shacl_report.tii	existe	17991	2026-05-31T20:19:13.660Z	Ver
tmp_pytest/web_runtime_report.json	no existe	0	-	Ver
tmp_pytest/web_workflow_plan.json	existe	1964	2026-05-31T20:17:32.584Z	Ver
tmp_pytest/live_dcat_catalog.jsonld	existe	11485	2026-05-31T20:20:35.588Z	Ver
tmp_pytest/live_dcat_validation_report.tii	existe	18054	2026-05-31T20:20:36.497Z	Ver
tmp_pytest/workflow_first_plan.json	existe	76	2026-05-31T20:20:11.023Z	Ver
tmp_pytest/workflow_second_plan.json	existe	76	2026-05-31T20:20:13.431Z	Ver
ttn_ingestor/ttn_ingestor/resources/shaclmanifest.json	existe	5161	2026-04-13T17:28:20.061Z	Ver
ttn_ingestor/config/operational_profile.yaml	existe	344	2026-05-12T17:17:41.431Z	Ver

Figura F.9: Pantalla *Artefactos*: listado de evidencias con vista previa (*Ver*) y apertura nativa (*Abrir*) para HTML renderizado y PDF en el visor del navegador. Elaboración propia.

F.10. Historial de ejecuciones

La pantalla *Ejecuciones* lista todos los *jobs* generados por la consola en orden cronológico inverso. Cada entrada muestra el identificador único del *job*, la operación ejecutada, el estado (*ok* o *error*), la duración en segundos y el código de salida. El operario puede abrir cualquier *job* para ver el log completo, el resumen del resultado y los artefactos que produjo, lo que permite reconstruir la trazabilidad de cualquier ejecución pasada sin necesidad de volver a ejecutarla.

Operacion	Estado	Creado	Salida	Log
Suite completa	correcto	2026-05-31T20:44:46.236Z	0	Ver
Suite completa	correcto	2026-05-31T20:23:55.572Z	0	Ver
Validar DCAT live	correcto	2026-05-31T20:20:22.596Z	0	Ver
Suite completa	correcto	2026-05-31T20:19:54.273Z	0	Ver
Validar DCAT	correcto	2026-05-31T20:19:12.339Z	0	Ver
Exportar DCAT	correcto	2026-05-31T20:18:24.300Z	0	Ver
Exportar DCAT	correcto	2026-05-31T20:18:16.873Z	0	Ver
Aplicar workflow	correcto	2026-05-31T20:17:40.256Z	0	Ver
Dry-run del workflow	correcto	2026-05-31T20:17:31.696Z	0	Ver
Comprobar pre-requisitos	correcto	2026-05-31T20:14:52.023Z	0	Ver
Validar DCAT live	correcto	2026-04-20T17:19:41.953Z	0	Ver
Validar DCAT live	correcto	2026-04-20T17:18:38.471Z	0	Ver
Vaciar PostgreSQL demo en OpenMetadata	correcto	2026-04-20T15:33:22.871Z	0	Ver
Validar DCAT live	correcto	2026-04-20T15:18:49.876Z	0	Ver
Validar DCAT live	correcto	2026-04-20T15:19:15.438Z	0	Ver
Aplicar workflow	correcto	2026-04-20T15:18:07.011Z	0	Ver
Dry-run del workflow	correcto	2026-04-20T15:18:02.777Z	0	Ver

Figura F.10: Pantalla *Ejecuciones*: historial de *jobs* con estado, duración, código de salida y acceso al log y artefactos. Elaboración propia.

F.11. Federación del catálogo

El archivo `dcatalog.jsonld` es RDF válido conforme a DCAT-AP-ES 1.0.0 con perfil HVD, comprobado por SHACL. A partir de ese punto el operario puede llevarlo a cualquiera de los destinos compatibles del ecosistema español o europeo de datos abiertos.

- **Harvester CKAN con extensión `ckanext-dcat`:** subir el JSON-LD a través del endpoint `/dataset/import` del portal destino. CKAN reconoce las clases DCAT y crea los datasets correspondientes.
- **Portal nacional `datos.gob.es`:** el archivo cumple los requisitos publicados de DCAT-AP-ES y puede ofrecerse al equipo de federación como entrada del proceso editorial del portal.
- **Triplestore RDF (Apache Jena Fuseki o equivalente):** cargar el catálogo en un *dataset* del *store* y consultarlo con SPARQL para auditoría o integración con otras fuentes RDF.
- **rdflib en Python:** cargar localmente el grafo con `rdflib.Graph().parse('dcatalog.jsonld', format='json-ld')` para análisis programático.

El listado F.2 ilustra cómo se entregaría el archivo a un harvester compatible.

```
1 curl -X POST https://harvester.example.org/api/dataset/import \
2   -H "Authorization: Bearer $TOKEN" \
```

```
3 -H "Content-Type: application/ld+json" \  
4 --data-binary @tmp_pytest/dcat_catalog.jsonld
```

Código F.2: Envío del catálogo a un harvester CKAN con extensión DCAT.

La plataforma, por tanto, no se limita a producir un JSON-LD formalmente válido: lo entrega listo para integrarse con la cadena editorial real que se utiliza hoy en datos.gob.es, data.europa.eu y los portales basados en CKAN.

F.12. Resumen del recorrido

Paso	Pantalla / acción	Artefacto producido	Sección que lo formaliza
1	Infraestructura — comprobar	Estado de pods	§ 5.3
2	Ingesta — revisar	Tablas gold en OM	§ 5.4
3	Gobierno — editar + guardar	gold_governance.csv	§ 5.6
4	Workflow — dry-run	Plan JSON	§ 5.5
5	Workflow — apply	Cambios aplicados en OM	§ 5.5
6	DCAT — exportar	dcat_catalog.jsonld	§ 3.3
7	Validación — suite	Informe consolidado JSON	cap. 6
8	SHACL — manifiesto congelado	manifest.json (informe .ttl en Artefactos)	§ 3.6
9	Artefactos — descargar / Abrir	Catálogo RDF, informe HTML/PDF	§ F.9
10	Ejecuciones — revisar log	Historial de <i>jobs</i> trazables	§ F.10

Tabla F.1: Mapa del recorrido demo: acción del operario, artefacto producido y sección de la memoria donde se justifica formalmente. Elaboración propia.

Este recorrido es la demostración mínima defendible de la plataforma: parte de un entorno reproducible, opera íntegramente desde la consola web sin abandonar el navegador, recorre las nueve etapas con artefactos versionados y termina con un archivo RDF listo para entrar en la cadena editorial real de datos abiertos.

Referencia bibliográfica

- [1] Universidad de Castilla-La Mancha. Ficha oficial del Trabajo Fin de Estudios: Diseño y configuración de un modelo de metadatos en OpenMetadata conforme al estándar DCAT-AP para la interoperabilidad de catálogos de datos. https://tfe.apps.uclm.es/ges_mi_tfe.aspx, 2026. Portal institucional de gestión del Trabajo Fin de Estudios de la UCLM (acceso autenticado). Copia literal conservada localmente en docs/tfe_ficha_oficial_uclm.txt.
- [2] W3C. Data Catalog Vocabulary (DCAT) - Version 3. <https://www.w3.org/TR/vocab-dcat-3/>, 2024. W3C Recommendation; última consulta: mayo de 2026.
- [3] Iniciativa Aporta. Guía técnica y modelo DCAT-AP-ES 1.0.0. <https://datosgob.es/github.io/DCAT-AP-ES/>, 2025. Última consulta: mayo de 2026.
- [4] David Puig and Juan Mañes. El potencial uso de la metodología de DAMA en la gestión de los datos abiertos. <https://datos.gob.es/es/blog/el-potencial-uso-de-la-metodologia-de-dama-en-la-gestion-de-los-datos-abiertos>, 2021. Publicado en datos.gob.es; última consulta: mayo de 2026.
- [5] DAMA International. DAMA-DMBOK: Data Management Body of Knowledge. <https://dama.org/learning-resources/dama-data-management-body-of-knowledge-dmbok/>, 2026. Última consulta: mayo de 2026.
- [6] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010. ISBN 978-0984521401.
- [7] Project Management Institute. A Guide to the Project Management Body of Knowledge (PMBOK Guide), 7th Edition, 2021. ISBN 978-1628256642.

-
- [8] GitHub, Inc. About Projects (GitHub Projects). <https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [9] W3C. RDF 1.1 Concepts and Abstract Syntax. <https://www.w3.org/TR/rdf11-concepts/>, 2014. W3C Recommendation; última consulta: mayo de 2026.
- [10] W3C. JSON-LD 1.1. <https://www.w3.org/TR/json-ld11/>, 2020. W3C Recommendation.
- [11] SEMIC – European Commission. DCAT Application Profile for Data Portals in Europe (DCAT-AP). <https://semiceu.github.io/DCAT-AP/releases/2.1.1/>, 2023. Versión 2.1.1; última consulta: mayo de 2026.
- [12] Iniciativa Aporta. Validación DCAT-AP-ES. <https://datosgob.es.github.io/DCAT-AP-ES/validation/>, 2025. Última consulta: mayo de 2026.
- [13] W3C. Shapes Constraint Language (SHACL). <https://www.w3.org/TR/shacl/>, 2017. W3C Recommendation.
- [14] Collate. OpenMetadata Documentation. <https://docs.open-metadata.org/>, 2026. Documentación oficial de OpenMetadata; última consulta: mayo de 2026.
- [15] Anjana Data. Gobierna tus datos abiertos conforme a DCAT-AP-ES: listo para publicar en datos.gob.es. <https://anjanadata.com/gobierna-tus-datos-abiertos-conforme-a-dcat-ap-es-listo-para-publicar-en-datos-gob-es/>, 2026. Referencia de mercado consultada en mayo de 2026.
- [16] DataHub Project. DataHub: A Metadata Platform for the Modern Data Stack. <https://datahubproject.io/docs/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [17] Amundsen (LF AI & Data Foundation). Amundsen: data discovery and metadata engine. <https://www.amundsen.io/amundsen/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [18] Docker, Inc. Docker overview. <https://docs.docker.com/get-started/docker-overview/>, 2026. Documentación oficial; última consulta: mayo de 2026.
-

- [19] Kubernetes SIGs. kind – Kubernetes IN Docker. <https://kind.sigs.k8s.io/docs/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [20] The Kubernetes Authors. Kubernetes Concepts. <https://kubernetes.io/docs/concepts/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [21] Helm. Using Helm. https://helm.sh/docs/intro/using_helm/, 2026. Última consulta: mayo de 2026.
- [22] Meta Platforms. React – The library for web and native user interfaces. <https://react.dev/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [23] Vercel. Next.js Documentation. <https://nextjs.org/docs>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [24] Microsoft. TypeScript Handbook. <https://www.typescriptlang.org/docs/handbook/intro.html>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [25] pytest team. pytest – helps you write better programs. <https://docs.pytest.org/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [26] Vitest team. Vitest – Next Generation Testing Framework. <https://vitest.dev/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [27] Gobierno de España. Norma Técnica de Interoperabilidad de Reutilización de Recursos de Información del Sector Público (NTI-RISP). <https://datos.gob.es/es/doc-tags/norma-tecnica-de-interoperabilidad-nti-risp>, 2013. Resolución de 19 de febrero de 2013, BOE 4 de marzo de 2013; última consulta: mayo de 2026.
- [28] Mermaid Contributors. Mermaid – Diagramming and charting tool. <https://mermaid.js.org/intro/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [29] Databricks. What is a Medallion Architecture? <https://www.databricks.com/glossary/medallion-architecture>, 2024. Documentación oficial; última consulta: mayo de 2026.
- [30] The PostgreSQL Global Development Group. PostgreSQL Documentation. <https://www.postgresql.org/docs/>, 2026. Documentación oficial; última consulta: mayo de 2026.

-
- [31] Python Software Foundation. `dataclasses` – Data Classes (Python Standard Library). <https://docs.python.org/3/library/dataclasses.html>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [32] RDFLib Team. RDFLib – pure Python package for working with RDF. <https://rdflib.readthedocs.io/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [33] Ashley Sommer. `pySHACL` – Python validator for SHACL. <https://github.com/RDFLib/pySHACL>, 2026. Repositorio oficial; última consulta: mayo de 2026.
- [34] Comisión Europea. Reglamento de Ejecución (UE) 2023/138 relativo a conjuntos de datos de alto valor (HVD). <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32023R0138>, 2023. Diario Oficial de la Unión Europea; última consulta: mayo de 2026.
- [35] Comisión Europea – DG DIGIT, Interoperability Test Bed. SEMIC SHACL Validator. <https://www.itb.ec.europa.eu/shacl/semic-shacl/upload>, 2026. Servicio oficial de validación SHACL del Interoperability Test Bed (ITB); última consulta: mayo de 2026.
- [36] OpenMetadata. Kubernetes Deployment. <https://docs.open-metadata.org/v1.11.x/deployment/kubernetes>, 2026. Última consulta: mayo de 2026.